CS336 · Spring 2026 · Lecture 16

# 后训练（二）：可验证奖励与推理模型

## 如何阅读这份自学增强讲义

原始英文幻灯片完整保留；每组原页之后紧接对应中文讲解。书签可跳转到原页与讲解。页码中的“原页”始终指原始 PDF 页码。

中文部分为自学补充；课堂转述、原论文结论与编者推导应按文中标记区分。第三方保存的字幕仅用于检索定位，不等同于教师审校逐字稿。

学习目标：独立推导序列策略梯度、状态基线、GAE 的望远镜求和与组内归一化；解释 PPO/GRPO 的采样、概率比和 KL；比较 R1、Kimi、Qwen 的训练链，并能判断长度增长、测试时预算和奖励上升究竟支持什么结论。

先修：条件概率、期望、方差、对数导数、softmax/交叉熵、自动微分和语言模型自回归分解；建议先理解上一讲的 SFT、偏好奖励与 DPO。符号主线：x 是题目，y 是回答，πθ 是当前策略，πold 或 $\pi_i$ 是本轮采样旧策略，πref 是参考策略，R 是任务奖励，V 是前缀值函数，Â 是优势估计，G 是同题采样条数。

阅读主线：原页1–4提出奖励问题；5–16从 PPO 公式走到实现；17–24解释 GRPO 及归一化偏差；25–38学习 R1 与蒸馏；39–48学习 Kimi 的目标、长度和系统；49–54学习 Qwen3 的小题库 RL 与思考预算；55–60扩展到代码 agent 与奖励投机；61收束。每组保留全部原页，再接相邻中文说明。

证据与来源索引：slides 为官方 Spring 2026 第16讲，已于2026-10-15通过HTTP下载确认与本地原稿逐字节一致；ppo、alpaca、nano用于对应算法/代码核对；math、dr、r1、kimi、s1、qwen、coder为原论文或作者报告。带“编者推导/补充/例子”的内容是为自学加入的计算和解释，不冒充教师原话。captions 是第三方字幕，仅在已核对的时间范围作课堂定位；课件简写与论文不一致处保留原页并说明。

使用方式：先读原图的坐标、图例和公式，再用相邻讲解复算数字，最后回答文末自测。所有模型成绩均按原报告的版本与协议解释；这里没有重新训练模型，也没有复现原论文评测。公式中的 clip、stop-gradient、EOS/padding 和长度 reduction 必须一起读。

# Lecture 16

## Post-training 2

## Reinforcement learning from verifiable rewards

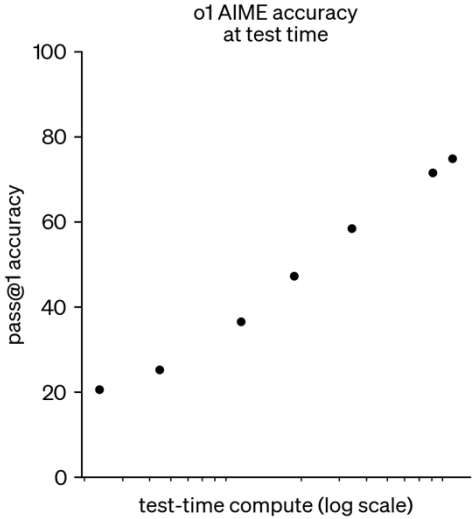
CS336

# The class thus far

Pre-training + RLHF gets you to ~ GPT3.5… **Today we'll get all the way to o1/r1**

OMG! What Will Happen When A.I. Makes BuzzFeed Quizzes?

The site announced plans to use tools from the creator of ChatGPT to create content. Former quiz writers aren't surprised.




ChatGPT: Optimizing Language Models for Dialogue

o1

第 16 讲 · 原页 1–2 · 中文讲解

# 从偏好对齐走向可验证任务

## 本讲要回答的问题

上一讲的监督微调（SFT）教模型如何回答，偏好对齐让回答更符合评价者的选择。本讲问的是：如果能够自动判断一个解答是否成功，能否用大量尝试和反馈进一步训练模型？RLVR 是 reinforcement learning from verifiable rewards，即利用可验证奖励的强化学习。它描述奖励来源，不指定优化算法；PPO、GRPO 都可用于不同奖励来源。反过来，GRPO 也能优化偏好模型的分数，不能把算法名称当作奖励可靠性的保证。

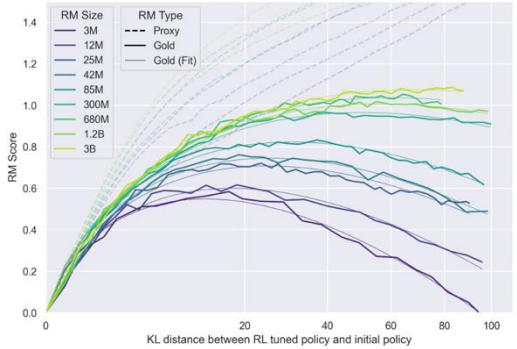
## 读原页的两张图

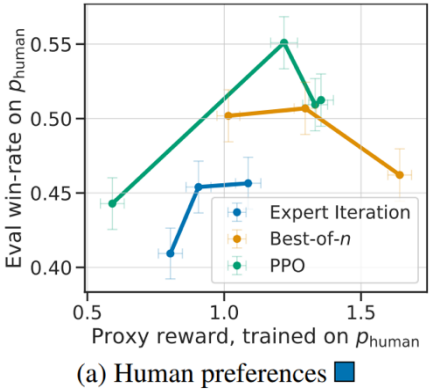
第2页左侧代表通用对话模型，右侧图的横轴是对数刻度的测试时计算量，纵轴是特定 AIME 评测的正确率。图要引出的现象是：给已经训练好的模型更多解题计算，可能提高该评测上的成绩。它不是“任何模型多输出文字都更聪明”的定律。训练计算改变参数；测试时计算在参数固定时用于生成、检查或搜索。后面应分别追踪这两个预算，而不是把长回答直接等同于训练进步。

本节参考（完整书目见讲义末尾）：[slides]

# The goal – expand the scope and power of RL

With RLHF: we can't cleanly scale out due to overoptimization

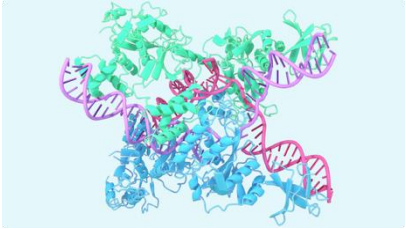



(a) Human preferences

Can we work in domains where RL excels? Where we optimize *exactly* what we want

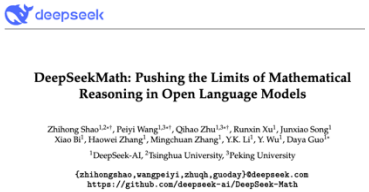


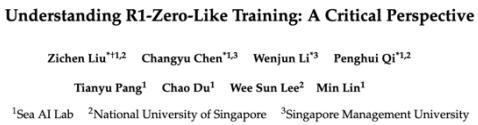
# The lecture today

## 1. Core algorithms..

PPO → GRPO

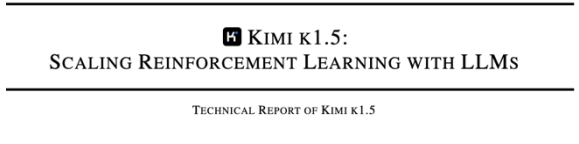



GRPO variants




## 2. Case studies






第 16 讲 · 原页 3–4 · 中文讲解

# 奖励的可验证性：能自动打分，不等于目标没有漏洞

## 偏好奖励与可验证奖励的区别

偏好奖励常由人类比较回答，再训练 rφ(x,y) 近似“哪一个更好”；它是对复杂目标的代理，训练分布之外可能把冗长、迎合或特定措辞错当质量。数学最终答案比对、程序运行测试、形式证明检查则直接检验一个较清晰的条件。这样的反馈通常更廉价、重复性更强，能支持大规模在线采样。但“可验证”是对某个规格而言：几个测试全过，不代表程序在所有输入上正确；答案相同，不代表推导每一步有效；形式证明通过，也只证明了所形式化的命题。

## 读图与反例

第3页的曲线把代理奖励、真实评价与偏离初始策略的程度放在一起，说明继续提高代理分数可能同时降低外部评价。编者例子：若代码奖励只检查 f(2)=4，恒返回4也得满分；若奖励要求最终整数等于42，一段错误推导碰巧输出42仍可得分。因此要区分真实任务效用 U、验证器输出 R，以及训练目标 E[R]。只有 R 在实际探索到的输出上仍与 U 一致，优化 R 才可靠。

## 阅读路线与边界

先从策略梯度走到 PPO，再把值函数替换成同一道题的组内比较；接着分析标准差与长度归一化改变了什么；最后比较 R1、Kimi、Qwen 的数据、奖励和系统。第3页 AlphaGo、蛋白质与 Lean 的图是“反馈结构更清楚”的动机示意，不代表这些领域用了同一套语言模型 RL 配方。第60页的代码智能体会再次展示：即便有可执行测试，模型也可能绕过任务条件。

本节参考（完整书目见讲义末尾）：[slides]

# Recap: PPO in theory

**A brief (and high level) intro to the various ideas in PPO..**

**Attempt 1**: Policy gradients (variances are too high)

$$\nabla_\theta E_{p_\theta}[R(z)] = E_{p_\theta}[\mathrm{R(z)}\nabla_\theta \log p_\theta\,(z)]$$

**Attempt 2**: TRPO (Linearize the problem around the current policy)

$$\begin{aligned} \underset{\theta}{\text{maximize}} \quad & \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t\right] \\ \text{subject to} \quad & \hat{\mathbb{E}}_t[\mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]] \leq \delta. \end{aligned}$$

**Attempt 3:** PPO (Clip the ratios at some eps)

$$L(s, a, \theta_k, \theta) = \min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}A^{\pi_{\theta_k}}(s,a),\ \ \mathrm{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1-\epsilon, 1+\epsilon\right)A^{\pi_{\theta_k}}(s,a)\right)$$

第 16 讲 · 原页 5–5 · 中文讲解

# 编者推导：从离散答案到策略梯度

## 先固定符号与成立条件

令 x 是从固定训练分布 D 抽取的题目，$y=(y_1,\ldots,yT)$ 是完整回答，$\pi\theta(y|x)=\prod_t \pi\theta(y_t|x,y<t)$，R(x,y) 是整条回答的奖励。把 EOS 计入生成动作，T 可以因回答而异。暂不加 KL 和长度惩罚，目标为 $J(\theta)=E_x E_y{\sim}\pi\theta[R(x,y)]$。这里假设 R 不显式依赖 θ，概率可微且可以交换求和与微分；若使用硬截断或受限采样，还必须明确实际采样分布。

## 逐步得到 **score-function** 梯度

对固定 x，$J_x=\Sigma_y \pi\theta(y|x)R(x,y)$。于是 $\nabla J_x=\Sigma_y R(x,y)\nabla\pi\theta(y|x)$。利用 $\nabla\pi=\pi\nabla\log\pi$，得到 $\nabla J_x=E_y[R(x,y)\nabla\log\pi\theta(y|x)]$。再由乘积取对数，$\log\pi\theta(y|x)=\Sigma_t \log\pi\theta(y_t|x,y<t)$，得到 $\nabla J=E[R\Sigma_t\nabla\log\pi\theta(y_t|x,y<t)]$。验证器无需可微：梯度穿过的是“生成这个答案的概率”，而非通过离散答案反传到验证器。样本奖励和采样结果在这一步都视为常量。

## 手算一个最小例子

设模型只会输出正确答案 C 或错误答案 W，$p=\pi\theta(C)=\text{sigmoid}(\theta)$，R(C)=1、R(W)=0，则 J=p，真实导数是 p(1−p)。当 p=0.25 时，正确样本的 log概率导数为1−p=0.75，错误样本为−p=−0.25。原始 REINFORCE 只在抽到 C 时有梯度，因此期望为0.25×0.75=0.1875。减去固定基线 b=0.25 后，C 的梯度为0.75×0.75=0.5625，W 的梯度为(−0.25)×(−0.25)=0.0625，期望仍为0.1875。一个错误样本也能帮助降低错误动作概率；这说明基线改变单次更新，却可保持期望梯度。

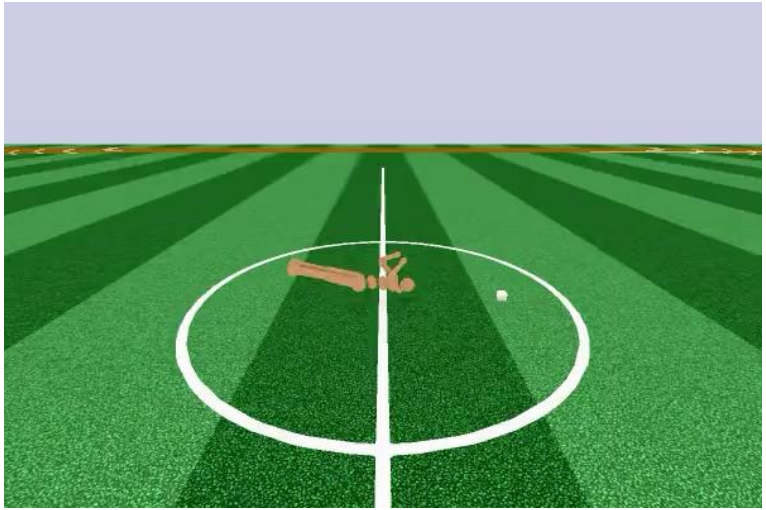
## 从这一步到 **PPO**

原页把高方差、TRPO 和 PPO 排成三个尝试，但它们不只解决同一个问题：基线主要控制采样噪声；TRPO 还要控制策略更新后分布偏移，使用旧策略附近的代理目标与 KL 约束；PPO 用便宜的一阶裁剪近似控制过大的更新。高奖励样本少时，即便公式正确，也可能很久采不到有用探索。

本节参考（完整书目见讲义末尾）：[slides] · [ppo]

# PPO

Policy gradient method for optimizing rewards in actual RL tasks..



From the PPO announcement blog (2017)



OpenAI Five (2019)

# PPO – at a conceptual level

The objective is basically all there is to it at a conceptual level

**Algorithm 1** PPO-Clip

1: Input: initial policy parameters $\theta_0$, initial value function parameters $\phi_0$
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
4: Compute rewards-to-go $\hat{R}_t$.
5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function $V_{\phi_k}$.
6: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left( \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

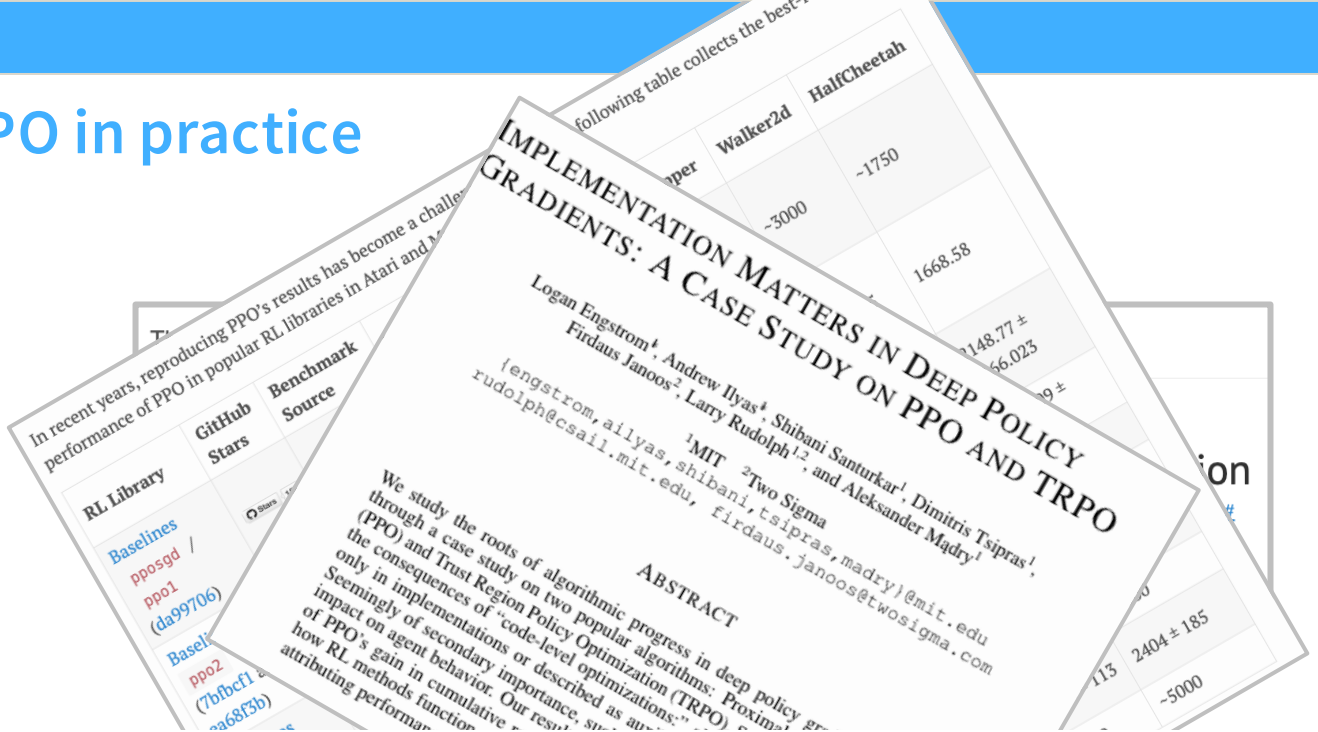
typically via stochastic gradient ascent with Adam.
7: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left( V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.
8: **end for**

[From - https://spinningup.openai.com/en/latest/algorithms/ppo.html]

# PPO in practice




We need to look at a live implementation when talking about PPO

第 16 讲 · 原页 6–8 · 中文讲解

# PPO：裁剪限制更新动机，工程决定实际行为

## 由旧策略样本构造代理目标

令 πold 是采集这一批轨迹的策略，$\rho_t=\pi\theta(a_t|s_t)/\pi\text{old}(a_t|s_t)$，$\hat{A}_t$ 是固定的优势估计。最大化 $L=E\text{old}[\min(\rho_t\hat{A}_t,\text{clip}(\rho_t,1-\varepsilon,1+\varepsilon)\hat{A}_t)]$。ρ 把旧样本处的动作概率变化纳入优化；在 token 级实现里，这仍是局部代理目标，不是对整条新策略轨迹分布的完全重要性校正。πold 在一轮内保持固定，更新后的 πθ 才会在下一轮收集新轨迹。

## 正负优势分别手算

取 ε=0.2。若 Â=2，ρ=1.5 时两项为3和2.4，取最小值2.4，继续增大这个好动作的概率不再增加此样本的目标。若 Â=−2，ρ=0.5 时两项为−1和−1.6，取−1.6，继续降低坏动作概率也不再额外获益。但若 Â=−2 且 ρ=1.5，取−3，会继续惩罚把坏动作变得更常见。裁剪针对的是有利方向上的过度更新动机，不是把实际概率强制锁进区间；共享参数、其他样本与优化器仍可让比值越界。

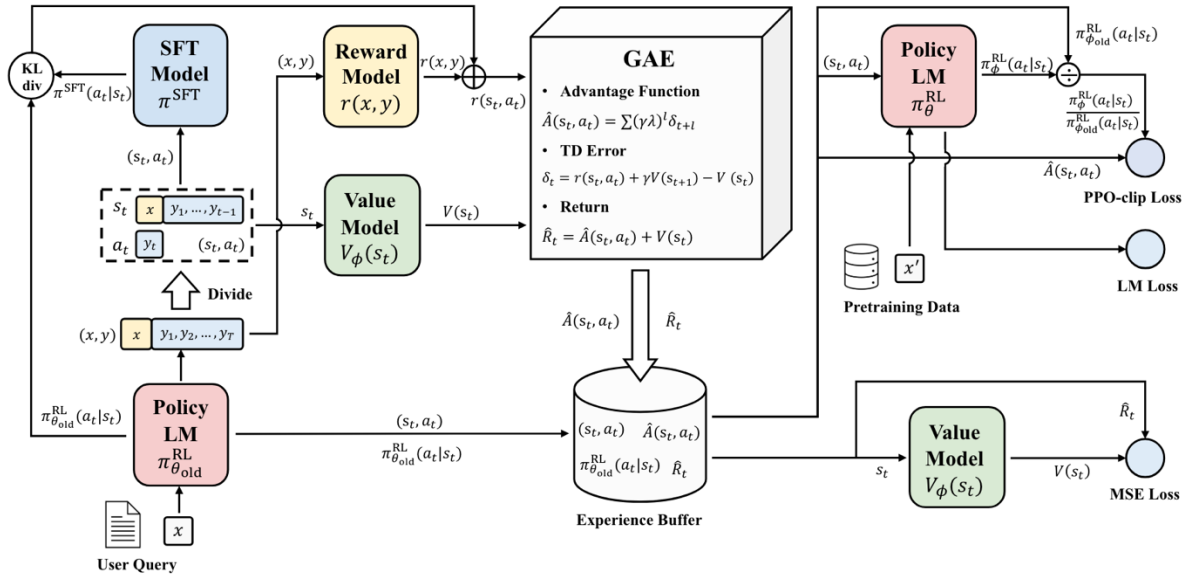
## 读第7页伪代码与第8页警示

一次迭代包括采样、算回报与优势、数次策略梯度更新，以及用回报拟合值函数。值函数 Vφ(s) 预测当前状态之后的收益，不是判定整条回答质量的 reward model。第6页机器人与游戏说明 PPO 的应用历史；第8页“实现细节影响效果”的拼图提醒：优势白化、值损失权重、mask、奖励缩放、早停、优化器和采样温度会共同决定行为。公式相同不保证代码等价。KL 早停和相对参考模型的 KL 奖励也要分清：前者看新旧策略的本轮变化，后者约束长期偏离一个参考策略。

本节参考（完整书目见讲义末尾）：[slides] · [ppo]

# PPO – idealization (?) for language models

In the language model setting..




Pretty similar to the RL formulation. Actions operate over tokens, big dense reward at the very end operating on full sequence

[From Zheng et al 2023]

# PPO – let's look at an implementation

Implementation – AlpacaFarm's PPO

| Method | Simulated Win-rate (%) | Human Win-rate (%) |
|---|---|---|
| GPT-4*† | $79.0 \pm 1.4$ | $69.8 \pm 1.6$ |
| ChatGPT*† | $61.4 \pm 1.7$ | $52.9 \pm 1.7$ |
| PPO | $46.8 \pm 1.8$ | $55.1 \pm 1.7$ |
| DPO | $46.8 \pm 1.7$ | - |
| Best-of-1024 | $45.0 \pm 1.7$ | $50.7 \pm 1.8$ |
| Expert Iteration | $41.9 \pm 1.7$ | $45.7 \pm 1.7$ |
| SFT 52k | $39.2 \pm 1.7$ | $40.7 \pm 1.7$ |

Used by several others to reasonable success

SALMON: SELF-ALIGNMENT WITH INSTRUCTABLE REWARD MODELS

**Zhiqing Sun**[1,2*] **Yikang Shen**[1] **Hongxin Zhang**[3] **Qinhong Zhou**[3] **Zhenfang Chen**[1]

**David Cox**[1] **Yiming Yang**[2] **Chuang Gan**[1,3]

[1]MIT-IBM Watson AI Lab, IBM Research
[2]Language Technologies Institute, CMU
[3]UMass Amherst

ALIGNING LARGE MULTIMODAL MODELS WITH FACTUALLY AUGMENTED RLHF

**Zhiqing Sun**[*♠], **Sheng Shen**[*♣], **Shengcao Cao**[*◇]
**Haotian Liu**[♡], **Chunyuan Li**[‡], **Yikang Shen**[△], **Chuang Gan**[†▽△], **Liang-Yan Gui**[†◇]
**Yu-Xiong Wang**[†◇], **Yiming Yang**[†♠], **Kurt Keutzer**[†♣], **Trevor Darrell**[†♣]
♣UC Berkeley, ♠CMU, ◇UIUC, ♡UW–Madison, ▽UMass Amherst
‡Microsoft Research, △MIT-IBM Watson AI Lab

第 16 讲 · 原页 9–10 · 中文讲解

# 把语言模型放进强化学习回路

## 图中四种模型各做什么

状态 $s_t=(x,y<t)$，动作 $a_t=y_t$；生成 token 后，状态接上该 token。策略模型负责生成；参考模型 πref 常取冻结的 SFT 模型，用于约束偏离；奖励模型或验证器给完整回答打分；值模型预测各前缀之后的回报。策略与值模型可以共享部分参数，也可以分开，不能仅凭四个逻辑角色推断一定有四套完整权重。图中的经验缓存保存本轮 rollout 供数次更新，不等于可以无限重用陈旧数据。

## 稀疏终局信号如何影响每个 **token**

原页9写“big dense reward at the very end”，这里的 dense 可理解为对整段答案综合打分；在时间轴上，若只有结束时才给任务分数，它仍是稀疏终局奖励。令 $r_t=0$（$t<T$）、$rT=R$，则没有折扣时每个前缀的 reward-to-go 都是 R；它们的优势仍可不同，因为 $V(s_t)$ 不同。用 token 状态可以做信用分配；把整条输出作为单一动作又可视为 contextual bandit。两种表示不矛盾，但与工具和外部环境交互的 agent 不能简单套成一次静态答案。

## 第**10**页表格的正确读法

AlpacaFarm 用模拟偏好和人类偏好比较不同方法，为后续展示 PPO 实现提供实例。表中 win-rate 不是数学正确率，也不能直接与后面的 AIME pass@1 比较。应先问评判者、参考对手、样本集合与误差区间，再解释 PPO、DPO、best-of-n 或 expert iteration 的差异。

本节参考（完整书目见讲义末尾）：[slides] · [alpaca]

# PPO in practice

PPO outer loop. Invoke an inner loop to optimize the loss over some rollouts.

```python
def step_with_rollouts(self, rollouts):
    """Based on fixed rollouts, run PPO for multiple epochs."""
    assert isinstance(self.optimizer, AcceleratedOptimizer), (
        "`optimizer` must be pushed through `accelerator.prepare`. "
        "Otherwise the `accelerator.accumulate` context manager won't correctly disable `zero_grad` or `step`."
    )
    rollouts_dataloader = self.get_rollouts_dataloader(rollouts=rollouts)
    stats_list = []
    for epoch_idx in range(self.args.noptepochs):
        for batch_idx, rollouts_batch in tqdm.tqdm(
            enumerate(rollouts_dataloader, 1), disable=not self.accelerator.is_main_process, desc="gradstep"
        ):
            with self.accelerator.accumulate(self.policy):
                ppo_loss, stats_for_this_step = self.compute_loss(rollouts_batch)
                self.accelerator.backward(ppo_loss)
                if self.accelerator.sync_gradients:
                    # Gradient norm almost blows up at some point, but stabilizes eventually, even w/o clipping.
                    if self.args.max_grad_norm is not None:
                        self.accelerator.clip_grad_norm_(self.policy.parameters(), self.args.max_grad_norm)
                    stats_for_this_step["loss/grad_norm"] = self._compute_grad_norm()
                    stats_list.append(stats_for_this_step)
                self.optimizer.step()
                self.optimizer.zero_grad(set_to_none=True)
    return common.merge_dict(stats_list, torch.stack)  # list of dict -> dict: str -> 1-D tensor
```

https://github.com/tatsu-lab/alpaca_farm/blob/30717ddae735365de756ee2085191b491a71788d/src/alpaca_farm/rl/ppo_trainer.py

# PPO in practice – loss computation

AlpacaFarm code – loss computation. Pretty standard stuff.

```python
class PPOTrainer(rl_trainer.RLTrainer):

    def compute_loss(self, rollouts: Dict[str, Tensor]) -> Tuple[Tensor, Dict]:
        values, old_logprob, returns, advantages, queries, query_attn_masks, responses = common.prepare_inputs(
            common.unpack_dict(
                rollouts,
                keys=("values", "logprobs", "returns", "advantages", "queries", "query_attn_masks", "responses"),
            ),
            device=self.accelerator.device,
        )
        outputs = self.policy(queries, query_attn_masks, responses, temperature=self.args.temperature)

        vpred = outputs["values"]
        vpredclipped = torch.clamp(
            vpred,
            min=values - self.args.cliprange_value,
            max=values + self.args.cliprange_value,
        )
        vf_losses1 = (vpred - returns) ** 2.0
        vf_losses2 = (vpredclipped - returns) ** 2.0
        vf_loss = 0.5 * torch.maximum(vf_losses1, vf_losses2).mean()
        vf_clipfrac = (vf_losses2 > vf_losses1).to(torch.get_default_dtype()).mean()

        logprob = outputs["logprobs"]
        ratio = torch.exp(logprob - old_logprob)
        # When current policy is close to the old policy, the KL component of this advantage is approximately correct.
        pg_losses = -advantages * ratio
        pg_losses2 = -advantages * torch.clamp(ratio, min=1.0 - self.args.cliprange, max=1.0 + self.args.cliprange)
        pg_loss = torch.maximum(pg_losses, pg_losses2).mean()
        pg_clipfrac = (pg_losses2 > pg_losses).to(torch.get_default_dtype()).mean()  # noqa

        loss = pg_loss + self.args.vf_coef * vf_loss
```

$$L(s,a,\theta_k,\theta) = \min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}A^{\pi_{\theta_k}}(s,a),\ \ \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1-\epsilon, 1+\epsilon\right)A^{\pi_{\theta_k}}(s,a)\right)$$

Cliprange=0.2

第 16 讲 · 原页 11–12 · 中文讲解

# 沿 PPO 代码追踪张量和损失

## 外层采样，内层重算概率

第11页的外层使用固定 rollout，再通过数据加载器切 minibatch 做数次优化。缓存 logprobs 是旧策略在当时真实生成 token 上的 log 概率；第12页重新前向得到当前 logprob，ratio=exp(logprob−old_logprob)。不能每个梯度步都把 old_logprob 覆盖成当前值，否则比值总为 1，裁剪失去参照。梯度累积是在模拟更大 batch；梯度范数裁剪是另一种稳定手段，与 PPO ratio 裁剪不同。

## 编者补充：形状与符号检查

有 B 条回答、补齐到 T 个 token 时，old_logprob、logprob、value、advantage 与 mask 通常均为 [B,T]；它们已经沿词表维收集了实际采样 token 的值，不是 [B,T,Vocab]。有效回答 token 的 mask 应排除提示词和 padding，并按约定包含 EOS。最大化 min surrogate 在最小化代码里写成 max(−Âρ,−Âclip(ρ))，这是负号变换。还要确认 reduction 是每条回答先平均、全体 token 平均，还是固定常数归一化；三者在不同长度上并不等价。

## 值损失为何也裁剪

原页同时显示 values 的新预测、旧预测，以及回报 targets。值函数通过平方误差逼近回报；其裁剪分支限制相对旧预测的大幅变化，随后与未裁剪误差取较保守的一项。策略损失加上加权值损失才是这段代码的总损失。值函数学得好可以降低优势噪声，学得不好则会引入噪声或偏差；这个额外训练任务与显存成本正是后面 GRPO 想去掉的部分。

本节参考（完整书目见讲义末尾）：[slides] · [alpaca]

# PPO in practice – rollouts.

```python
    def rollout(self, queries_data) -> Dict[str, Tensor]:
        """Rollout trajectories with policy.

        Args:
            queries_data: Sequence of batches or DataLoader.
                Each batch is a dict with keys 'queries' and 'query_attn_masks'.

        Returns:
            Dictionary with keys
                'queries', 'query_attn_masks', 'responses',
                'logprobs', 'ref_logprobs', 'values',
                'rewards', 'non_score_rewards', 'shaped_rewards'.
        """
        # Give up dropout throughout.
        self.policy.eval()
        self._make_fsdp_happy()
        # `keep_fp32_wrapper` retains the autocast wrapper of model.forward created by accelerate:
        #  recall one sets mixed precision options with accelerator.
        # The precise value of this arg doesn't matter here, since we use the unwrapped model only for respond.
        # Generally, try to use the wrapped model as much as you can, since it's got the autocast/cast-back wrappers.
        unwrapped_policy = self.accelerator.unwrap_model(self.policy, keep_fp32_wrapper=True)

        self.ref_policy.eval()
        self.reward_model.eval()

        rollouts = []
        for batch_idx, batch in tqdm.tqdm(
            enumerate(queries_data),
            disable=not self.accelerator.is_main_process,
            desc="rollout",
        ):
            # Sample rollouts.
            queries, query_attn_masks = common.unpack_dict(
                common.prepare_inputs(batch, device=self.accelerator.device),
                keys=("queries", "query_attn_masks"),
            )
            respond_outputs = unwrapped_policy.respond(queries, query_attn_masks, temperature=self.args.temperature)
            (responses,) = common.unpack_dict(respond_outputs, ("responses",))

            # Evaluate logprobs of the samples.
            rollouts_batch = {"queries": queries, "query_attn_masks": query_attn_masks, "responses": responses}
            policy_outputs = self.policy(**rollouts_batch, temperature=self.args.temperature)
            ref_policy_outputs = self.ref_policy(**rollouts_batch, temperature=self.args.temperature)
            policy_outputs = common.unpack_dict(
                policy_outputs, keys=("logprobs", "values", "entropies"), return_type=dict
            )
            ref_policy_outputs = common.unpack_dict(
                ref_policy_outputs, keys=("logprobs", "entropies"), return_type=dict
            )
            rollouts_batch.update(policy_outputs)
            rollouts_batch.update({f"ref_{key}": value for key, value in ref_policy_outputs.items()})

```

```python
            # Evaluate reward of the samples.
            text_queries, text_responses = tuple(
                self.tokenizer.batch_decode(tensor, skip_special_tokens=True, clean_up_tokenization_spaces=True)
                for tensor in (queries, responses)
            )
            del queries, responses  # Prevent mistakes.

            # We retokenizer, since policy and reward model might not have the same tokenizer.
            # TODO(lxuechen): Avoid retokenization when policy and reward tokenizer are the same.
            text_sequences = [q + r for q, r in utils.zip_(text_queries, text_responses)]
            # TODO(lxuechen): This response retokenization has issues with OPT, since the tokenizer always prepend
            #  <bos_token>. But the issue is local to post_reward, which isn't an issue if we don't penalize.
            sequences, responses = tuple(
                self.tokenizer(text, return_tensors="pt", padding=True, truncation=True)
                for text in (text_sequences, text_responses)
            )
            sequences, responses = common.prepare_inputs((sequences, responses), device=self.accelerator.device)

            reward_outputs = self.reward_model(**sequences)
            reward_outputs = self.post_reward(reward_outputs, responses.input_ids)
            rollouts_batch.update(reward_outputs)

            # Shape reward with KL penalty.
            shape_reward_outputs = self._shape_reward(
                rewards=rollouts_batch["rewards"],
                responses=rollouts_batch["responses"],
                logprobs=rollouts_batch["logprobs"],
                ref_logprobs=rollouts_batch["ref_logprobs"],
            )
            rollouts_batch.update(shape_reward_outputs)

            rollouts_batch_cpu = {key: value.cpu() for key, value in rollouts_batch.items()}
            rollouts.append(rollouts_batch_cpu)

        # Items in dict need to be of same shape.
        rollouts = common.merge_dict(rollouts, merge_fn=torch.cat)
        # Estimating advantages outside the loop gives more samples for reward normalization.
        advantages = self._estimate_advantage(
            rewards=rollouts["shaped_rewards"].to(self.accelerator.device),
            values=rollouts["values"].to(self.accelerator.device),
        )
        advantages = {key: value.cpu() for key, value in advantages.items()}
        return {**rollouts, **advantages}
```

# PPO in practice – reward shaping

**High level** – add per-token KL penalty, last-token full reward

**In practice?** Clip KL for sequences where new policy logp < reference logp

```
67      def _shape_reward(
68          self, rewards: Tensor, responses: Tensor, logprobs: Tensor, ref_logprobs: Tensor
69      ) -> Dict[str, Tensor]:
70          # For some reason, line below doesn't work.
71          # kl = (logits.softmax(dim=-1) * (logits.log_softmax(dim=-1) - ref_logits.log_softmax(dim=-1))).sum(dim=-1)
72          kl = torch.clamp(logprobs - ref_logprobs, min=0.0)
73          non_score_rewards = -self.kl_ctl.value * kl
74          shaped_rewards = non_score_rewards.clone()
75          # This introduces a small index off by one bug if pad_token_id == eos_token_id.
76          terminal_positions = (responses != self.tokenizer.pad_token_id).sum(dim=1) - 1
77          shaped_rewards[list(range(rewards.size(0))), terminal_positions] += rewards
78          return dict(shaped_rewards=shaped_rewards, non_score_rewards=non_score_rewards, kl=kl)
```

Helps with stability? If we blow up our model, this prevents kl from diverging

第 16 讲 · 原页 13–14 · 中文讲解

# Rollout、终止位置与 KL 奖励塑形

## 读 rollout 截图的顺序

先用策略采样回答；再在同一回答上计算策略与参考模型的 log 概率及值预测；解码后交给奖励模型评分；把终局分数与逐 token 正则项组合，最后计算优势。奖励模型可能使用不同 tokenizer，所以截图有重新编码步骤。题目与回答的边界、截断和 EOS 都可能改变奖励输入。第14页代码还直接提醒 pad 与 EOS 共用 ID 会导致终止位置偏一；自学时应理解这个漏洞，而不是照抄计数公式。

## 精确 KL、采样估计与截断项

在给定前缀 s 下，$D_{KL}(\pi||\pi_{ref})=\Sigma_a \pi(a|s)\log[\pi(a|s)/\pi_{ref}(a|s)]\geq 0$。若 a 真从 π 采样，单点 $\log\pi(a|s)-\log\pi_{ref}(a|s)$ 的期望等于该 KL，但单点值可以为负。编者例子：π=(0.8,0.2)，πref=(0.5,0.5)，两个 log 比为约0.4700和−0.9163；平均 KL 约0.1927。若先把负数截成0，期望变成约0.3760，已经不是同一个估计。

## 原代码的实际含义与边界

截图执行 clamp(logprobs−ref_logprobs,min=0)，针对采到的 token 裁掉负 log 比；再乘 −β 并在最后有效位置加任务奖励。原页说“clip KL for sequences”是概括，实际展示的是 token 张量操作。这是非负惩罚启发式，不能声称保留精确 KL 的无偏性，也不能保证阻止模型发散。理想的未截断版本可写 $r_t=-\beta\log(\pi_{old}/\pi_{ref})_t+1[t=T]R$。固定参考策略与采样旧策略职责不同，KL 系数、reward-to-go 和长度会共同决定总惩罚。

本节参考（完整书目见讲义末尾）：[slides] · [alpaca]

# PPO in practice – generalized advantage estimate

Instead of reward, we use advantages

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} := \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V \qquad \text{where} \qquad \delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t),$$

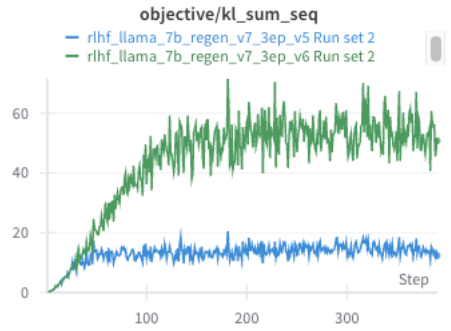
```python
def _estimate_advantage(self, rewards: Tensor, values: Tensor) -> Dict[str, Tensor]:
    """Generalized advantage estimation.

    Reference:
        https://arxiv.org/abs/1506.02438
    """
    if self.args.whiten_rewards:
        rewards = torch_ops.whiten(rewards, shift_mean=False)
    lastgaelam = 0
    advantages_reversed = []
    gen_length = self.args.response_len
    for t in reversed(range(gen_length)):
        nextvalues = values[:, t + 1] if t < gen_length - 1 else 0.0
        delta = rewards[:, t] + self.args.gamma * nextvalues - values[:, t]
        lastgaelam = delta + self.args.gamma * self.args.lam * lastgaelam
        advantages_reversed.append(lastgaelam)
    advantages = torch.stack(advantages_reversed[::-1], dim=1)
    returns = advantages + values
    advantages = torch_ops.whiten(advantages, shift_mean=True)
    return dict(returns=returns, advantages=advantages)
```

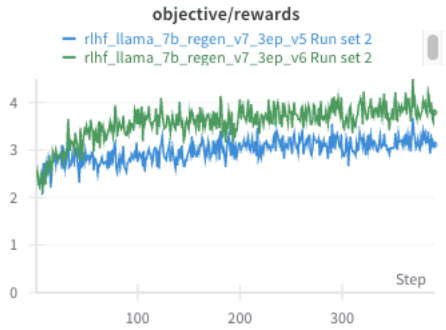
Funny detail – this is a bandit problem and gamma=lambda=1 works – this is the reward-to-go vs the value

# What do you expect to see in PPO?

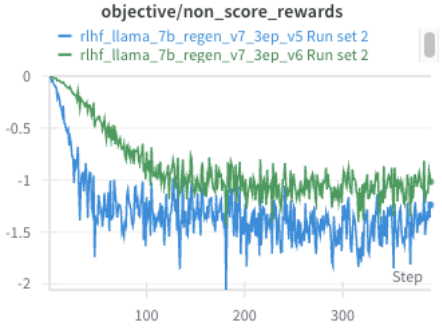
Increasing overall rewards




Incl. reward model




Negative KL rewards




This is a bandit setting, you expect reasonable training curves

第 16 讲 · 原页 15–16 · 中文讲解

# 优势、GAE 与训练曲线：把终局分数传回前缀

## 优势到底减去了什么

定义 $Q\pi(s,a)=E[$之后累计奖励$|s,a]$，$V\pi(s)=E_{a\sim\pi}[Q\pi(s,a)]$，优势 $A\pi=Q\pi-V\pi$。它衡量“这个动作比在同一状态下按当前策略平均行动好多少”。优势不是奖励本身，也不是模型置信度。定义实际回报 $G_t=\Sigma_{k\geq t}\gamma^{(k-t)}r_k$，可用 $G_t-V(s_t)$ 作蒙特卡洛优势；值函数仅作基线且当作常量时，误差不会自动变成同等大小的策略偏差，但它会改变方差；自举近似则还涉及估计偏差。

## GAE 的中间步骤

定义 TD 残差 $\delta_t=r_t+\gamma V(s_{t+1})-V(s_t)$，GAE 为 $\hat{A}_t=\Sigma_{l\geq 0}(\gamma\lambda)^l\delta_{t+l}$，也就是反向递推 $\hat{A}_t=\delta_t+\gamma\lambda\hat{A}_{t+1}$。$\gamma$ 控制未来奖励折扣，$\lambda$ 控制使用多远的自举误差信息；它们不是相同参数。对真实终止且 $V(sT+1)=0$ 的有限序列，$\gamma=\lambda=1$ 时逐项相加，$+V(s_{t+1})$ 与下一项 $-V(s_{t+1})$ 消掉，故 $\hat{A}_t=\Sigma_{k=t}^{T}r_k-V(s_t)$。到最大长度的截断若不是任务真终止，就不能未经说明把末端价值设为0。

## 手算三步与图表阅读

编者例子：$r=(0,0,1)$，$V=(0.2,0.4,0.6)$，终止价值0。$\gamma=\lambda=1$ 时 $\delta=(0.2,0.2,0.4)$，从后向前累加得 $\hat{A}=(0.8,0.6,0.4)$，加回 V 恰好得到回报 (1,1,1)。第16页三图分别是 KL 总量、任务奖励与非分数奖励；左图实际纵轴标为 objective/kl_sum_seq，不应照页上“overall rewards”的标题误认成总收益。KL 惩罚更负与奖励上升可以同时出现。应一起看 held-out 正确率、长度、终止率、KL、奖励均值与方差；训练分数上升本身不能排除奖励投机。

本节参考（完整书目见讲义末尾）：[slides] · [alpaca]

# Why do we need yet another RL algorithm..?

**Why not PPO?**

- In practice, complicated implementation
- Value model (memory hungry, involves additional tuning for training)

**Why not DPO?**

- Data not inherently pairwise (or in the form of Bradley-terry comparisons)
- Offline (though could be made online by iterating)

# New kid on the block: GRPO

**What's GRPO?**

- Start with PPO (many parts are similar)
- Remove the value function / advantage computation
- Calculate the advantage as "z-score within group"

**Group Relative Policy Optimization** In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question $q$, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model $\pi_\theta$ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \quad (1)$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \quad (2)$$

where $\varepsilon$ and $\beta$ are hyper-parameters, and $A_i$ is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{std(\{r_1, r_2, \cdots, r_G\})}. \quad (3)$$

(PPO for reference)

$$\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}A^{\pi_{\theta_k}}(s,a),\ \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1-\epsilon, 1+\epsilon\right)A^{\pi_{\theta_k}}(s,a)\right)$$

**In the *online* case (rollout+immediate update), this is just policy gradient with group normalized rewards**

第 16 讲 · 原页 17–18 · 中文讲解

# GRPO：用同题多次尝试替代值网络

## 为什么不直接用 PPO 或 DPO

PPO 的值函数需要参数、激活、优化器状态和额外调参。DPO 的典型训练资料是偏好对与参考策略，经典形式是离线优化；它并非不能迭代在线收集数据，只是可验证任务天然提供每条尝试的分数，不一定有现成的成对偏好标签。因此 GRPO 的吸引力是把在线探索保留下来，同时减少值函数这条训练分支，而不是证明 PPO 或 DPO 原则上不能解题。

## 写出同一组究竟是什么

给定一个 x，从 $\pi_{old}$ 独立采样 G 条回答 $y_i$，记奖励 $R_i$、组均值 $m=(1/G)\Sigma_i R_i$、组标准差 s。终局奖励版本常令 $\hat{A}_i=(R_i-m)/(s+\varepsilon_{num})$，并把同一个 $\hat{A}_i$ 广播到该回答的所有有效 token。策略项为 $(1/G)\Sigma_i(1/T_i)\Sigma_t \min(\rho_{it}\hat{A}_i, clip(\rho_{it},1-\varepsilon_{clip},1+\varepsilon_{clip})\hat{A}_i)$，再减去按实现约定聚合的 βKL。数值稳定常数 $\varepsilon_{num}$ 与裁剪半径 $\varepsilon_{clip}$ 不是一回事。第18页展示的是更简略的回答级写法；第23页才显式突出 token 和长度归一化。

## “就是策略梯度”的适用范围

当采样后立即更新且 $\theta=\theta_{old}$ 时，$\rho=1$，裁剪通常不活跃，局部策略梯度形如 $\hat{A}\nabla\log\pi$。若每条回答又除 $T_i$，它对应的是带该权重的梯度，并不自动等于原始序列期望奖励的无偏梯度。多步重复优化后，比值、clip 和 KL 都会产生实质影响。去掉 critic 不等于没有优势估计，而是改用组内相对奖励估计；省下值网络显存的同时，每题 G 次生成仍可能很贵。

本节参考（完整书目见讲义末尾）：[slides] · [math] · [r1]

# GRPO is very simple (thanks to lack of value function..)

**You can** (and people do) write tiny GRPO implementations

- Compute reward for each rollout
- Mean/Var normalization per group
- Compute KL term
- Gradient updates on the loss

We can walk through this example from https://github.com/McGill-NLP/nano-aha-moment

```python
def compute_pg_loss(
    policy_model: Union[DeepSpeedEngine, PreTrainedModel],
    reference_model: Union[DeepSpeedEngine, PreTrainedModel],
    batch: Dict[str, torch.Tensor],
    total_response_len: int,
) -> Tuple[torch.Tensor, Dict[str, float]]:
    """
    Compute the policy gradient loss with KL penalty between policy and reference models.

    This function:
    1. Computes log probabilities for both policy and reference models
    2. Calculates KL divergence penalty between the models
    3. Computes policy gradient loss using advantages
    4. Combines the losses with KL coefficient

    Args:
        policy_model: The model being trained
        reference_model: The reference model for KL penalty calculation
        batch: Dictionary containing:
            - input_ids: Tensor of shape [batch_size, seq_len]
            - attention_mask: Tensor of shape [batch_size, seq_len]
            - labels: Tensor of shape [batch_size, seq_len] with -100 for ignored positions
            - advantages: Tensor of shape [batch_size, seq_len]

    Returns:
        Tuple containing:
            - loss: Combined policy gradient and KL penalty loss (scalar tensor)
            - metrics: Dictionary with detailed loss components:
                - policy_loss: Pure policy gradient loss
                - kl_penalty: KL divergence penalty
                - entropy: Policy entropy
    """
    input_ids = batch["input_ids"]  # [batch_size, seq_len]
    attention_mask = batch["attention_mask"]  # [batch_size, seq_len]
    labels = batch["labels"]  # [batch_size, seq_len]
    advantages = batch["advantages"]  # [batch_size, seq_len]

    model_inputs = {
        "input_ids": input_ids,
        "attention_mask": attention_mask,
        "labels": labels,
    }

    labels_mask = (labels[..., 1:] != -100).float()  # [batch_size, seq_len-1]

    with torch.no_grad():
        ref_logps = compute_token_log_probs(
            reference_model, model_inputs, TEMPERATURE
        )  # [batch_size, seq_len-1]

    logps = compute_token_log_probs(policy_model, model_inputs, TEMPERATURE)  # [batch_size, seq_len-1]

    kl_penalty = torch.exp(ref_logps - logps) - (ref_logps - logps) - 1  # [batch_size, seq_len-1]
    kl_penalty = kl_penalty * labels_mask  # [batch_size, seq_len-1]

    entropy = -logps.sum() / labels_mask.sum()  # scalar

    policy_loss = -logps * advantages[..., 1:]  # [batch_size, seq_len-1]
    policy_loss = policy_loss * labels_mask  # [batch_size, seq_len-1]

    loss = (policy_loss + KL_COEFFICIENT * kl_penalty).sum() / total_response_len  # scalar
```

# Advantage computation is also very simple..

Basically just the ‘vanilla’ GRPO setup.

Main difference here is just the 1e-4 stability factor in the std calculation

```
    assert len(all_generations) == len(all_finish_reasons)
    assert len(all_generations) == len(samples) * GENERATIONS_PER_SAMPLE

    # Process responses and calculate rewards
    groups = [
        list(range(i, i + GENERATIONS_PER_SAMPLE))
        for i in range(0, len(all_generations), GENERATIONS_PER_SAMPLE)
    ]  # example: [[0, 1, 2], [3, 4, 5], [6, 7, 8]]

    all_query_token_ids, all_responses_token_ids, all_advantages = [], [], []

    stats = {
        "response_lengths": [],
        "rewards": [],
        "non_stop_rate": [],
    }

    for sample, group_indices in zip(samples, groups):
        finish_reasons = [all_finish_reasons[i] for i in group_indices]
        response_token_ids = [all_generations[i] for i in group_indices]
        responses = tokenizer.batch_decode(response_token_ids, skip_special_tokens=False)

        rewards_and_metrics = [compute_reward(resp, sample) for resp in responses]
        rewards, reward_metrics = zip(*rewards_and_metrics)

        rewards = np.array(rewards) # [group_size]
        response_advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-4)

        advantages = [
            [resp_adv] * len(resp)
            for resp_adv, resp in zip(response_advantages, response_token_ids)
        ]

        all_query_token_ids.extend([sample["input_ids"]] * GENERATIONS_PER_SAMPLE)
        all_responses_token_ids.extend(response_token_ids)
        all_advantages.extend(advantages)

        stats["rewards"].extend(rewards)
        stats["non_stop_rate"].extend([fr != "stop" for fr in finish_reasons])
        stats["response_lengths"].extend([len(ids) for ids in response_token_ids])
        for rm in reward_metrics:
            for k, v in rm.items():
                stats.setdefault(f"reward_metrics/{k}", []).append(v)

    episodes = {
        "all_query_token_ids": all_query_token_ids,
        "all_response_token_ids": all_responses_token_ids,
        "all_advantages": all_advantages,
    }

    return episodes, stats
```

第 16 讲 · 原页 19–20 · 中文讲解

# 一个可手算的 GRPO batch

## 从组内分数到 token 损失

编者例子：同题四条回答的奖励为 (1,0,0,0)，m=0.25。若使用总体标准差，$s=\sqrt{[(0.75^2+3\times0.25^2)/4]}=\sqrt{0.1875}\approx0.4330$，因此优势约为 (1.7321,−0.5774,−0.5774,−0.5774)。若使用样本标准差，分母改为 G−1，s=0.5，优势变为 (1.5,−0.5,−0.5,−0.5)。所以读第20页 torch.std 风格代码时，要核对 correction/unbiased 参数与框架版本，不能把不同约定算出的数值硬说成相同。

## 零方差与探索缺失

如果四次全错或全对，所有 $R_i-m$ 都为0，加入1e−4只避免除零，不会创造正确性梯度。G=1 时还要注意某些样本标准差实现会产生 NaN。对于二元奖励、独立尝试、单次成功概率 p，一组同时含成功和失败的概率是1−p^G−(1−p)^G。p=0.1、G=4 时仅为0.3438；大多数组没有相对正确性信号。增加 G、选择适当难度、保持采样多样性，才可能提高有效组比例，但都会消耗预算或改变训练分布。

## 实现阅读清单与 KL 细节

第19页的最小实现展示生成、评分、组内归一化、KL 与策略更新。概念上要冻结参考概率、detach 奖励与优势、只对回答 token 算损失、确认同题分组正确。设 $u=\pi_{ref}(a|s)/\pi_\theta(a|s)$，常见非负 KL 估计为 u−logu−1≥0；a 从 πθ 采样时它的期望等于 $D_{KL}(\pi_\theta||\pi_{ref})$。如果样本来自旧策略且重复更新，就不能不加条件地宣称该样本平均仍精确等于当前 KL；采样分布、估计值与实际反向传播表达式要分开看。

## 编者补充：探索、熵与长度预算一起设计

采样温度升高通常让分布更平坦，熵 $H(\pi)=-\Sigma_a\pi(a)\log\pi(a)$ 增大时可探索更多动作，但更多随机 token 也可能破坏已经掌握的解法。训练越偏向少数高奖励轨迹，越可能过早失去多样性；熵奖励或数据课程可以缓解，但必须明确它们改变了目标。提高温度、top-k/top-p 截断与增加 G 不是同一件事：前两者改变采样分布，后者增加尝试数。缓存的 old log概率应与实际采样策略一致，不能随意用另一个温度的概率声称同策略更新。

更长的生成预算给检查、回溯和工具反馈留出空间，也增大验证器被试探的机会与计算成本；更长并不自动提高路径多样性，重复同一错误只是花更多 token。若模型开始利用验证漏洞，高组内奖励还会放大这种行为。因此应同步看独立正确率、奖励差异组比例、采样熵、重复率、超长/截断率与验证器误判，避免只靠增加长度或随机性来声称解决了探索。

本节参考（完整书目见讲义末尾）：[slides] · [nano] · [math]

# How well does it work?

## GRPO from the original paper

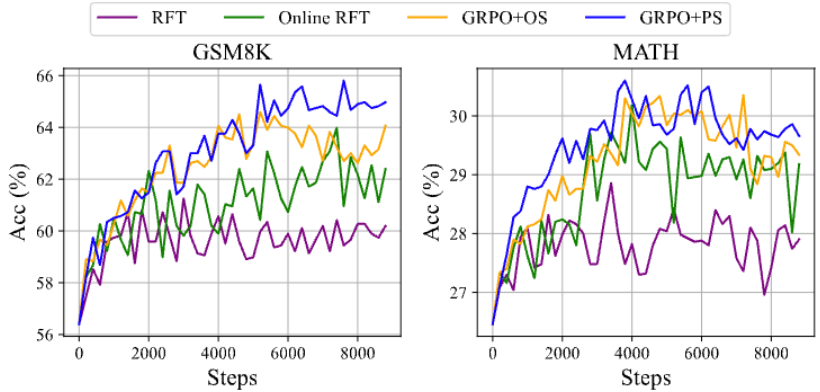



Figure 5 | Performance of the DeepSeekMath-Instruct 1.3B model, which was further trained using various methods, on two benchmarks.

Outperforms RFT (reinforcing correct answers),
with some gains from process supervision

We will get back to this later..

# Thinking carefully about the GRPO objective..

The key difference in GRPO vs PPO: the advantage

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}.$$

Is this good? A minor RL detour..

**Baselining**: We can subtract any state-dependent term from our rewards

## 13.4 REINFORCE with Baseline

The policy gradient theorem (13.5) can be generalized to include a comparison of the action value to an arbitrary *baseline* $b(s)$:

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a \Big(q_\pi(s,a) - b(s)\Big) \nabla \pi(a|s,\boldsymbol{\theta}). \tag{13.10}$$

The baseline can be any function, even a random variable, as long as it does not vary with $a$; the equation remains valid because the subtracted quantity is zero:

$$\sum_a b(s) \nabla \pi(a|s,\boldsymbol{\theta}) \;=\; b(s) \nabla \sum_a \pi(a|s,\boldsymbol{\theta}) \;=\; b(s) \nabla 1 \;=\; 0.$$

The policy gradient theorem with baseline (13.10) can be used to derive an update rule using similar steps as in the previous section. The update rule that we end up with is a new version of REINFORCE that includes a general baseline:

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \Big(G_t - b(S_t)\Big) \frac{\nabla \pi(A_t|S_t,\boldsymbol{\theta}_t)}{\pi(A_t|S_t,\boldsymbol{\theta}_t)}. \tag{13.11}$$

Sutton and Barto

第 16 讲 · 原页 21–22 · 中文讲解

# 结果监督、过程监督与合法基线

## 第21页的曲线支持多强的结论

DeepSeekMath 图比较 RFT、在线 RFT、GRPO 的结果监督（OS）与过程监督（PS）。横轴为训练步数，纵轴为 GSM8K 或 MATH 准确率；在这些实验设置里 GRPO 优于只保留正确回答做拟合的方案，过程监督还带来一些增益。RFT 可理解为采样后筛选正确答案，再作监督学习；PS 则需要对中间推理片段给分。不能把这张图当作“PS 一定必要”或“任何模型 GRPO 都最好”的证据，后面的 R1 正好没有采用过程监督。

## 编者推导：为何可减去状态基线

固定状态 s，若 b(s) 与本次动作 a 无关，$E_a[b(s)\nabla\log\pi(a|s)]=b(s)\Sigma_a\pi(a|s)\nabla\log\pi(a|s)=b(s)\Sigma_a\nabla\pi(a|s)=b(s)\nabla1=0$。因此 $E[(Q-b)\nabla\log\pi]$ 与 $E[Q\nabla\log\pi]$ 相同。条件是相对于当前动作具有这种独立性，并且在策略损失中不沿 b 反向传播；不是“任意从当前答案算出的数都能随便减”。基线可以是 prompt 函数或前缀值函数；取 V 往往有助减小噪声，但最小方差基线一般还受 score 梯度范数影响，不能宣称 V 总是唯一最优。

## 减法与除法为何不是同一件事

减去独立基线会让一个期望为0的项消失；除以某个与题目或样本有关的尺度，会重新加权梯度。即使 s 是固定的题目函数，1/s 也会改变不同题目对总目标的相对贡献；若 s 还依赖这组答案，随机比例与梯度相关，分析更复杂。下一节还要处理“组均值包含当前样本自身”这个经常被遗漏的有限组偏差。

本节参考（完整书目见讲义末尾）：[slides] · [math] · [dr]

# GRPO doesn't use a "valid" baseline

The division by the stdev term is not a valid baseline that preserves unbiasedness.

What is an unbiased-gradient version of GRPO?

**GRPO**

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\mathbf{o}_i|}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]\right\},$$

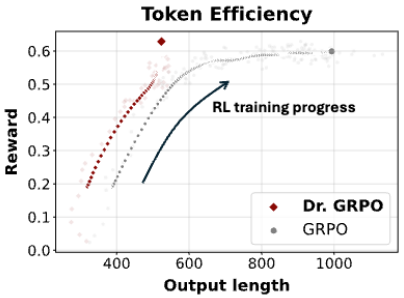
$$\text{where } \hat{A}_{i,t}=\frac{R(\mathbf{q},\mathbf{o}_i)-\text{mean}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})}{\text{std}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})}.$$

**Dr. GRPO**

GRPO Done Right (without bias)

$$\frac{1}{G}\sum_{i=1}^{G}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]\right\},$$

$$\text{where } \hat{A}_{i,t}=R(\mathbf{q},\mathbf{o}_i)-\text{mean}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\}).$$




Also note the modification on the length-normalizer term on the left..

Liu et al 2025. (this gets pretty close to reinforce w/ leave-one-out)

第 16 讲 · 原页 23–23 · 中文讲解

# 编者推导：组内均值、标准差与 Dr. GRPO 的偏差边界

## 包含自身的均值并非严格独立基线

固定 x，令 $g_i=\nabla\log\pi(y_i|x)$，组内回答条件独立同分布，$\mu=E[R]$。$E[g_i]=0$，且 $j\neq i$ 时 $E[R_jg_i]=E[R_j]E[g_i]=0$。于是 $E[m\ g_i]=(1/G)E[R_ig_i]$，所以 $E[(R_i-m)g_i]=(1-1/G)E[R_ig_i]$。不除标准差、不除样本长度时，组均值版本把期望梯度缩小了 $(G-1)/G$；固定 G 可视为统一尺度变化，却仍不应称严格无偏。G=4 对应0.75倍，G=1 则完全为0。

## 留一法如何修正

当 $G>1$，取 $b_{-i}=(1/(G-1))\Sigma_{j\neq i}R_j$，它不使用当前回答，条件独立保证 $E[b_{-i}g_i]=0$。并且 $R_i-b_{-i}=[G/(G-1)](R_i-m)$，可直接补偿上述缩放。第5页 p=0.25 的二元策略真梯度为0.1875；G=4 的包含自身均值估计期望为0.140625，留一法恢复0.1875。这是针对原始未裁剪、on-policy、固定奖励的 score 梯度结论；不能据此把后续 PPO 多步代理目标也叫无偏。

## Dr. GRPO 实际改动与原页措辞澄清

原页比较图把按实际 $T_i$ 的长度平均改成固定常数 L（如预设最大生成长度），并从优势中移除组标准差，仅保留 $R_i-m$。前者消除样本自身长度造成的额外权重，后者去掉难度相关的尺度变化。原页问“unbiased-gradient version”，应按这一比较语境理解：它去掉所讨论的两类偏置；图中仍是包含自身的组均值，所以严格期望仍有有限 G 的统一缩放。固定1/L也只是全局尺度，不是偏向某条长短回答；若要与未缩放的原目标梯度逐数值相等，还需恢复该尺度。不要把 Dr. GRPO 和留一法完全等同。

本节参考（完整书目见讲义末尾）：[slides] · [dr]

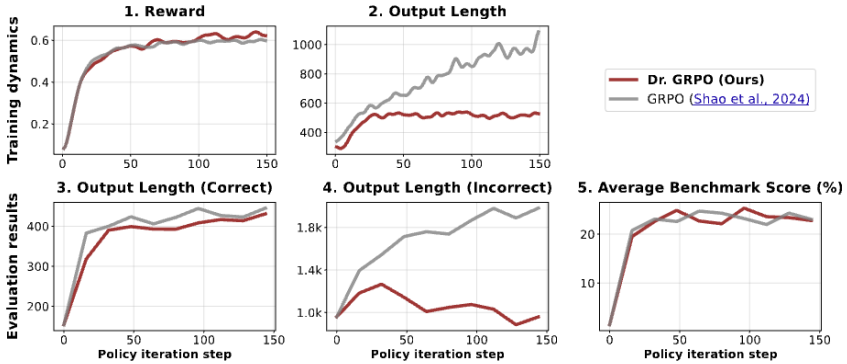
# Length biases of GRPO

What do these terms do? **Stdev** – upweights too easy or hard questions.

**Length normalization:**

**Response-level length bias**: This arises from dividing by $|\mathbf{o}_i|$. For positive advantages ($\hat{A}_{i,t} > 0$, indicating a correct response), this bias results in greater gradient updates for shorter responses, leading the policy to favor brevity in correct answers. Conversely, for negative advantages ($\hat{A}_{i,t} < 0$, indicating an incorrect response), longer responses are penalized less due to their larger $|\mathbf{o}_i|$, causing the policy to prefer lengthier responses among incorrect ones.

**What does the fix do?**



第 16 讲 · 原页 24–24 · 中文讲解

# 长度与难度归一化：被改变的到底是哪一份权重

## 为什么标准差会改变题目权重

对于二元奖励，成功概率为 p 时总体标准差为√[p(1−p)]。用它除梯度，会相对放大低方差的极易或极难题：p=0.5 时倒数为2，p=0.01 时约10.05。这里描述的是缩放因子，不是极难题必然贡献巨大有效更新；有限 G 时全错组常常没有梯度，经验标准差也未必接近总体值。还要区分同一题中罕见成功的巨大正优势，与不同题目在总梯度中的权重变化。

## 为什么按回答长度平均会引入方向性效果

在 Â 固定、clip 未活跃时，某条回答每个 token 的 score 梯度系数为 Â/T。两条长度2和8的回答若 Â=1，每 token 系数分别为0.5和0.125；若 Â=−1，则为−0.5和−0.125。于是短的好回答获得更强逐 token 奖励，长的坏回答受到较弱逐 token 惩罚。这里不能把不同 token 的梯度向量当成同方向数字相加，也不能宣称长回答总梯度范数一定小；结论是目标权重本身依赖采样长度，会影响训练诱因。

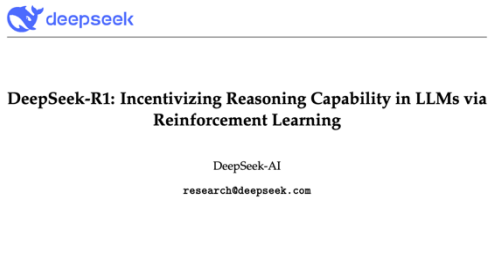
## 读修正后的曲线，不做单因素因果跳跃

第24页上方文字说明上述长度偏置，下方五图分别比较训练奖励、总体输出长度、正确输出长度、错误输出长度与平均测试成绩。Dr. GRPO 在该实验中减少了尤其是错误回答的长度增长，同时保持或改善成绩。第23页 token efficiency 图也强调类似的准确率可以对应不同输出成本。合理的结论是部分长度增长可能源于目标设计；不能反向推成“所有长推理都没用”。应在相同基座、数据、采样数和总生成 token 预算下做对照，报告正确与错误输出的长度分布。

本节参考（完整书目见讲义末尾）：[slides] · [dr]

# Case studies

Let’s look at a few representative open models with RLVR.


deepseek

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

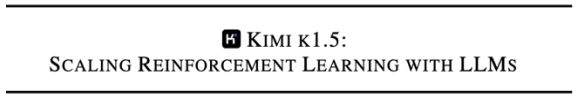
DeepSeek-AI

research@deepseek.com

**Deepseek R1**

Central to many recent RLVR efforts

Lots of interesting details


KIMI K1.5:
SCALING REINFORCEMENT LEARNING WITH LLMS

**Kimi K1.5**

Contemporaneous to R1, RLVR

Complementary details to R1


Qwen

2025-05-14

Qwen3 Technical Report

Qwen Team

https://huggingface.co/Qwen

https://modelscope.cn/organization/qwen

https://github.com/QwenLM/Qwen3

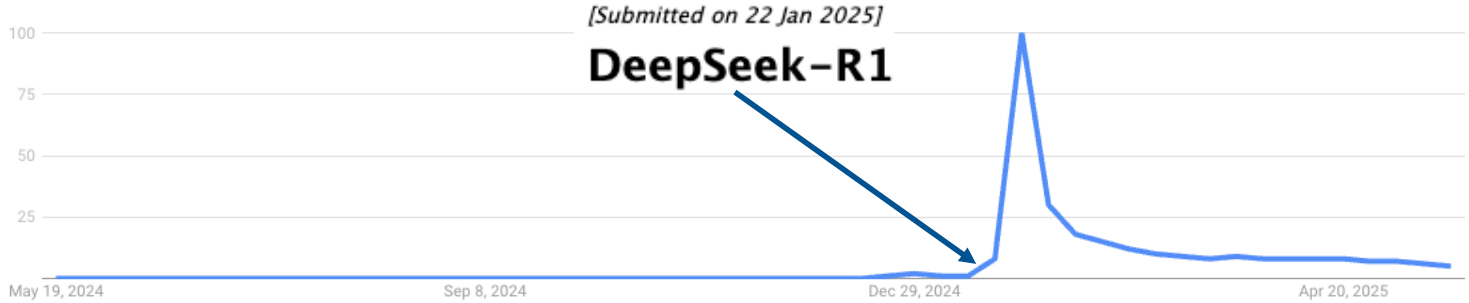
**Qwen 3**

Most recent open reasoning model attempt

Low-data RLVR

# Deepseek R1

The paper that launched a bit of a social phenomenon..




**What's remarkable about R1?**

- Performance exceeding OpenAI O1
- Open RL recipe (that is also pretty simple)
  - Ended speculations on the necessity of MCTS/PRMs
- SFT insights (both R1-zero and distil-r1)

第 16 讲 · 原页 25–26 · 中文讲解

# 案例阅读方法：把算法、数据、算力与评测拆开

## 为什么先看 **R1**

第25页列出 R1、Kimi K1.5、Qwen3 三条配方线索；第26页社会关注曲线说明 R1 的影响力，不是能力评测。它的重要性在于展示了公开可讨论的训练链，表明强推理模型不必一定采用显式蒙特卡洛树搜索（MCTS）或过程奖励模型（PRM）。这是“存在不用它们的成功方案”，不是“它们无效”的证明。

## 比较案例时固定四个问题

第一，起点是什么预训练模型，有没有 reasoning SFT？第二，训练题目如何筛选，奖励由规则、测试还是学习到的裁判给出？第三，每题采几条、允许多长、用哪种目标与 KL？第四，报告的是 pass@1、多次采样投票、还是不同工具框架下的表现？原页“超过 o1”等措辞都是当时指定表格中的局部比较；不同版本、预算和评测协议不能压成一个永久的强弱排序。这里按 Spring 2026 原课件的历史案例学习方法，不把标题里的“最新”当成今天的排行榜。

本节参考（完整书目见讲义末尾）：[slides] · [r1] · [kimi] · [qwen]

# Algorithm - GRPO

They build on the results of GRPO from DeepSeekMath..

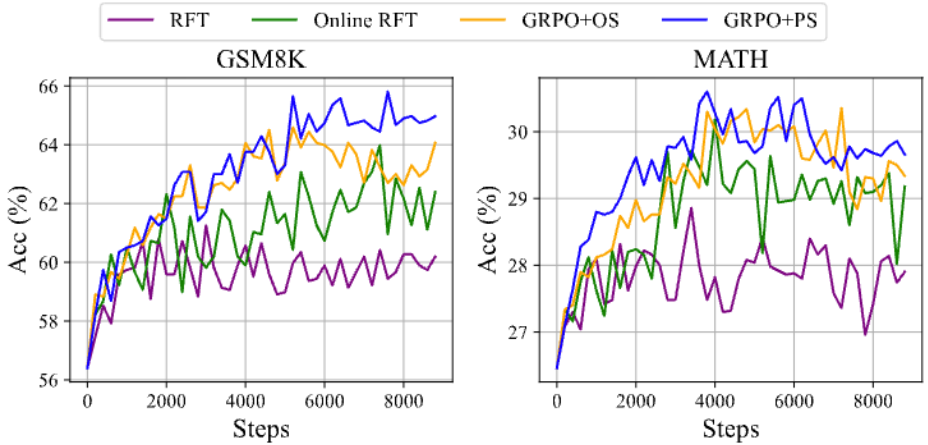



Figure 5 | Performance of the DeepSeekMath-Instruct 1.3B model, which was further trained using various methods, on two benchmarks.

But they do not use process supervision in R1 (more on this later..)

# Controlled setting – R1 zero.

**Setup:**

**Rewards**

- Accuracy rewards (is it correct?)
- Format rewards (use thinking tags)

**Data** (not public)

**Base model**: Deepseek-V3

**Results:** a bit worse than Openai O1

| Model | AIME 2024 | | MATH-500 | GPQA Diamond | LiveCode Bench | CodeForces |
|---|---|---|---|---|---|---|
| | pass@1 | cons@64 | pass@1 | pass@1 | pass@1 | rating |
| OpenAI-o1-mini | 63.6 | 80.0 | 90.0 | 60.0 | 53.8 | 1820 |
| OpenAI-o1-0912 | 74.4 | 83.3 | 94.8 | 77.3 | 63.4 | 1843 |
| DeepSeek-R1-Zero | 71.0 | 86.7 | 95.9 | 73.3 | 50.0 | 1444 |

第 16 讲 · 原页 27–28 · 中文讲解

# R1-Zero：不先做推理 SFT，直接探索可验证奖励

## 从 **DeepSeekMath** 到 **R1-Zero**

第27页再次出现 DeepSeekMath 的 OS/PS 曲线，作用是交代 GRPO 来源，不能把过程监督的增益自动归给 R1。R1-Zero 从 DeepSeek-V3-Base 出发，没有先加该实验的推理 SFT；“Zero”不意味着从随机权重训练，也不意味着没有预训练数据。它主要用正确性奖励与格式奖励，后者要求回答遵守思考与最终答案的结构。训练数据细节没有完整公开，所以配方公开不等于第三方可以逐样本复现。

## 格式奖励与正确性奖励分工

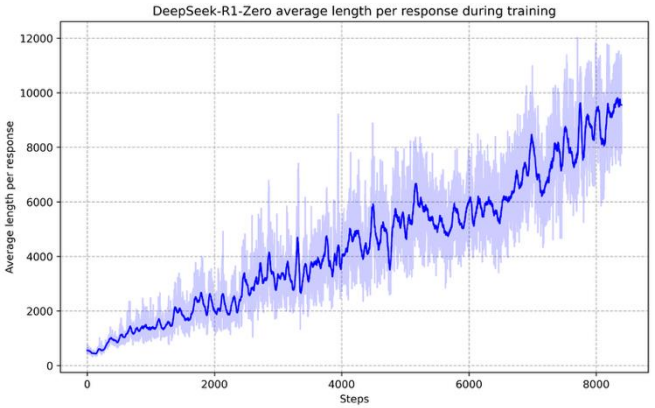
正确性奖励检验答案或代码测试；格式奖励让解析器能定位答案。两者相加时可能产生目标冲突：一个答案错但格式完美的输出可能得到非零总分。最终答案正确也不保证思考文本忠实反映内部计算，更不保证中间证明有效。应分别记录正确率与格式通过率，而不是只看总奖励。原页表中 R1-Zero 的 AIME2024 pass@1 为71.0，cons@64 为86.7，后者利用多次生成的一致性选择，不能与单次回答混比。

## 探索为什么依赖基座

编者补充：若正确解在当前策略下概率 p 很小，独立生成 n 次至少找到一次的概率是1−(1−p)^n。p=0.001、n=64 时约6.20%，即多数题仍得不到正例；p=0.1 时约99.88%。这里独立性只是便于理解的假设，实际输出相关性会降低收益。预训练提供语言、知识和解题片段，SFT 或提示可以提高初始命中率；RL 改变后续采样分布。有限预算下没发现解，不等于理论上永远无法发现。

本节参考（完整书目见讲义末尾）：[slides] · [r1]

# Interesting phenomena (?)




**Longer CoTs during training**

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a+x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a+x}} = x$, let's start by squaring both $\cdots$

$\left(\sqrt{a - \sqrt{a+x}}\right)^2 = x^2 \implies a - \sqrt{a+x} = x^2.$

Rearrange to isolate the inner square root term:

$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$

$\ldots$

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$

We started with the equation:

$\sqrt{a - \sqrt{a+x}} = x$

First, let's square both sides:

$a - \sqrt{a+x} = x^2 \implies \sqrt{a+x} = a - x^2$

Next, I could square both sides again, treating the equation: $\cdots$

$\ldots$

**'aha' moment?**

# But maybe a bit overstated

From some follow-up analyses in Dr. GRPO..

Length due to biased objective?

**GRPO**

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\mathbf{o}_i|}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]\right\},$$

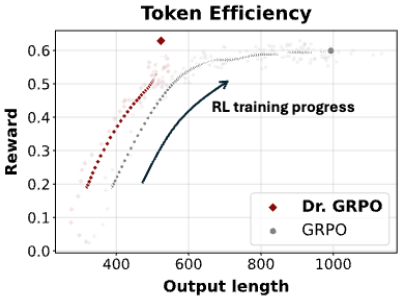
$$\text{where } \hat{A}_{i,t}=\frac{R(\mathbf{q},\mathbf{o}_i)-\text{mean}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})}{\text{std}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})}.$$

**Dr. GRPO**

GRPO Done Right (without bias)

$$\frac{1}{G}\sum_{i=1}^{G}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]\right\},$$

$$\text{where } \hat{A}_{i,t}=R(\mathbf{q},\mathbf{o}_i)-\text{mean}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\}).$$




Base model already has 'aha'

**Question(a):** Find the least positive integer $n$ such that $1/\sin 45^\circ \sin 46^\circ + 1/\sin 47^\circ \sin 48^\circ + \cdots + 1/\sin 133^\circ \sin 134^\circ = 1/\sin n^\circ$.

**Response**

First, I notice that the left-hand side of the equation has the form $\sum_{k=1}^{n} \frac{1}{\sin a_k \sin a_{k+1}}$, where $a_1 = 45^\circ, a_2 = 47^\circ, \ldots, a_{90} = 133^\circ$. I also notice that $\sin a_{k+1} = \sin(a_k + 2^\circ)$.

...

Simplifying, I get $\sin(3a) = 3\sin a - 4\sin^3 a$.

Aha! I can use this to get $\sin(a+2b) = \sin((a+b)+b) = 3\sin(a+b)$ ...

This looks messy, but I notice that it has the form $\sin a \cos b + \cos a \sin b$ again.

This gives me an idea. Maybe I can use this identity repeatedly to simplify the left-hand side of the equation.

...

第 16 讲 · 原页 29–30 · 中文讲解

# 长思考与“aha”：现象、机制与证据层级

## 原图分别展示了什么

第29页左图显示训练推进时平均响应长度增加，右侧是一条出现自我纠正措辞的样例。它们证明该训练设置里发生了长度和文本行为变化；单条样例不能证明模型突然获得一种全新算法，更不能据此断言文本描述的内省就是内部真实机制。第30页的跟进分析指出：某些长度增长可由目标权重解释，基座本身也能输出类似自我纠正的句子。两组观察可以同时成立。

## 怎样把“会说反思”与“反思有效”分开

编者补充：对相同题目，在固定生成 token 或时间预算下比较正确率；分别统计正确/错误回答长度、重复片段、有效修正次数；移除或替换反思句式，检查答案质量是否受损；检查基座在足够多样采样下的同类行为。若只增加“等等”却没有提高有效解题率，不能把它当成新能力证据。反过来，如果长度增长伴随相同预算下更高成功率，也不能仅因存在长度偏置就否认全部改进。

## 课堂补充 · **00:30:00–00:31:38**

本段字幕对应 R1-Zero 长度与 aha 展示，以及 Dr. GRPO 的反例。这里采用“作者观察到”“后续实验提示”等表述，保留两类证据之间的张力。后续提问 00:38:41–00:39:58 讨论长度归一化对正负优势的不同影响，以及总体长度增长由错误回答主导的实验观察。字幕是第三方保存的定位材料；上面的评测设计是编者补充，不是教师逐字原话。

本节参考（完整书目见讲义末尾）：[slides] · [r1] · [dr] · [captions]

# Pushing performance further – R1

**R1 vs R1-zero**

**Key differences:**

- SFT initialization
- Language consistency reward for CoT
- Non-verifiable rewards (in stage 2)



# SFT initialization..

> Unlike DeepSeek-R1-Zero, to prevent the early unstable cold start phase of RL training from the base model, for DeepSeek-R1 we construct and collect a small amount of long CoT data to fine-tune the model as the initial RL actor. To collect such data, we have explored several approaches: using few-shot prompting with a long CoT as an example, directly prompting models to generate detailed answers with reflection and verification, gathering DeepSeek-R1-Zero outputs in a readable format, and refining the results through post-processing by human annotators.

Start with plain ol (long) CoT, then maybe additional verification (vague)

- **Claimed Benefit**: interpretability
- **Note**: Origins on the data not quite clear..

第 16 讲 · 原页 31–32 · 中文讲解

# 完整 R1 的冷启动：先让探索进入可用区域

## 四阶段主线与一个容易漏掉的重置

第31页压缩成 V3→推理 SFT→GRPO→SFT/RLHF。展开后是：用少量长 CoT 做冷启动；进行推理强化学习；用强化学习后的模型生成并筛选数据，与非推理数据合并，对 DeepSeek-V3-Base 再做 SFT；最后进行面向多种任务的 RL。第三步并非只能理解为在同一条 RL 权重链上继续 SFT，原论文明确描述从基座开始拟合这批新数据。它体现了“模型生成训练资料，再训练另一个阶段”的数据迭代思想。

## 冷启动解决的是可读性、稳定性与探索起点

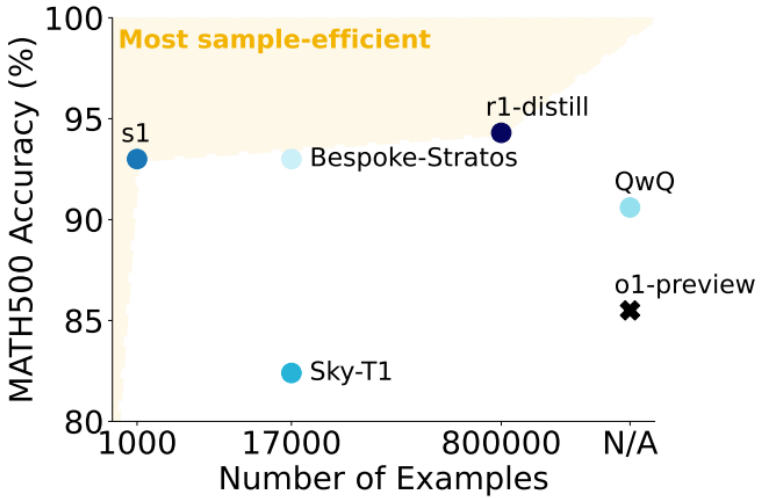
原页32的长段引文列出少样本长推理提示、直接生成带检查的答案、R1-Zero 输出整理和人工后处理等来源。小规模 SFT 可以把输出搬到更易读、易解析的区域，并改善初始 RL 不稳定；它不是凭空灌入全部知识，也不是对推理忠实性的证明。论文对完整数据来源与构造细节披露有限，中文注释应保留不确定性，不能擅自补成唯一的教师模型或固定数量。

本节参考（完整书目见讲义末尾）：[slides] · [r1]

# SFT for reasoning / math..

Even a small number of samples is effective for bootstrapping reasoning from LMs

| Model | # ex. | AIME 2024 | MATH-500 | GPQA Diamond |
|---|---|---|---|---|
| **API only** | | | | |
| o1-preview | N.A. | 44.6 | 85.5 | 73.3 |
| o1-mini | N.A. | 70.0 | 90.0 | 60.0 |
| o1 | N.A. | **74.4** | **94.8** | **77.3** |
| Gemini 2.0 Flash Think. | N.A. | 60.0 | N.A. | N.A. |
| **Open Weights** | | | | |
| Qwen2.5-32B-Instruct | N.A. | 26.7 | 84.0 | 49.0 |
| QwQ-32B | N.A. | 50.0 | 90.6 | 65.2 |
| r1 | ≫800K | **79.8** | **97.3** | **71.5** |
| r1-distill | 800K | 72.6 | 94.3 | 62.1 |
| **Open Weights and Open Data** | | | | |
| Sky-T1 | 17K | 43.3 | 82.4 | 56.8 |
| Bespoke-32B | 17K | **63.3** | **93.0** | 58.1 |
| s1 w/o BF | **1K** | 50.0 | 92.6 | 56.6 |
| **s1-32B** | **1K** | 56.7 | **93.0** | **59.6** |




1k Math and science questions + Long CoTs from Gemini / r1

# RL step

**The RL part is basically the same..**

Minor difference: additional language consistency loss

clear solutions. During the training process, we observe that CoT often exhibits language mixing, particularly when RL prompts involve multiple languages. To mitigate the issue of language mixing, we introduce a language consistency reward during RL training, which is calculated as the proportion of target language words in the CoT. Although ablation experiments show that such alignment results in a slight degradation in the model's performance, this reward aligns with human preferences, making it more readable. Finally, we combine the accuracy of reasoning tasks and the reward for language consistency by directly summing them to form the final reward. We then apply RL training on the fine-tuned model until it achieves convergence on reasoning tasks.

The note on language switching is interesting.. RL naturally leads to mixed languages?

第 16 讲 · 原页 33–34 · 中文讲解

# 少量推理 SFT 与语言一致性奖励

## 小数据为什么可能有效

第33页 s1 等模型的表格与散点图强调：已有强基座时，约千条精选长推理示范也可能显著改变输出策略。它不意味着从零训练只需要千条题目。图把示例数与 MATH500 成绩并列，模型、数据质量、生成预算和预算强制策略也不同，所以“样本效率高”不等于受控实验已证明只由样本量导致。s1 的结果还涉及测试时预算控制，不能把整张表的差异全部归于 SFT。

## 语言一致性是明确的多目标取舍

第34页在正确性之外加目标语言一致性分数，可按思考文本中目标语言词语比例计算。其作用是让输出更符合读者偏好；原论文也报告可能轻微损害性能。它是额外的可读性目标，不是数学正确性验证。中英混用可能来自多语预训练与当前优化压力，但不能因此写成“RL 必然导致混合语言”。还要定义词语分割、代码和数学符号如何统计，否则奖励本身存在边界。

## 编者例子：组合奖励可改变排序

若 Rtotal=Rcorrect+0.2Rlang，正确且语言一致性0.5的答案得1.1，错误但一致性1的答案得0.2；如果把语言权重提高到2，二者都得2。这说明奖励权重直接定义取舍，不能把格式、语言与正确性混成一个未经解释的“能力分”。实际 R1 的权重应以其披露为准，这组数字仅用于说明机制。

本节参考（完整书目见讲义末尾）：[slides] · [r1] · [s1]

# SFT/RLHF

The usual post-training process happens *after* RLVR / reasoning RL.

**SFT step, 2 epochs.**

Reasoning data – non-verifiable tasks (‘write a proof of X’), use V3 as a judge (600k)

Non-reasoning data – V3 SFT dataset (200k)

**RLHF step**

Re-use R1-zero style reasoning RLHF in here

Non-verifiable tasks - V3 RLHF pipeline. Still uses GRPO (for RLHF)

# How well does R1 work?

It’s pretty good (but you knew that)

| | Benchmark (Metric) | Claude-3.5-Sonnet-1022 | GPT-4o 0513 | DeepSeek V3 | OpenAI o1-mini | OpenAI o1-1217 | DeepSeek R1 |
|---|---|---|---|---|---|---|---|
| | Architecture | - | - | MoE | - | - | MoE |
| | # Activated Params | - | - | 37B | - | - | 37B |
| | # Total Params | - | - | 671B | - | - | 671B |
| English | MMLU (Pass@1) | 88.3 | 87.2 | 88.5 | 85.2 | **91.8** | 90.8 |
| | MMLU-Redux (EM) | 88.9 | 88.0 | 89.1 | 86.7 | - | **92.9** |
| | MMLU-Pro (EM) | 78.0 | 72.6 | 75.9 | 80.3 | - | **84.0** |
| | DROP (3-shot F1) | 88.3 | 83.7 | 91.6 | 83.9 | 90.2 | **92.2** |
| | IF-Eval (Prompt Strict) | **86.5** | 84.3 | 86.1 | 84.8 | - | 83.3 |
| | GPQA Diamond (Pass@1) | 65.0 | 49.9 | 59.1 | 60.0 | **75.7** | 71.5 |
| | SimpleQA (Correct) | 28.4 | 38.2 | 24.9 | 7.0 | **47.0** | 30.1 |
| | FRAMES (Acc.) | 72.5 | 80.5 | 73.3 | 76.9 | - | **82.5** |
| | AlpacaEval2.0 (LC-winrate) | 52.0 | 51.1 | 70.0 | 57.8 | - | **87.6** |
| | ArenaHard (GPT-4-1106) | 85.2 | 80.4 | 85.5 | 92.0 | - | **92.3** |
| Code | LiveCodeBench (Pass@1-COT) | 38.9 | 32.9 | 36.2 | 53.8 | 63.4 | **65.9** |
| | Codeforces (Percentile) | 20.3 | 23.6 | 58.7 | 93.4 | **96.6** | 96.3 |
| | Codeforces (Rating) | 717 | 759 | 1134 | 1820 | **2061** | 2029 |
| | SWE Verified (Resolved) | **50.8** | 38.8 | 42.0 | 41.6 | 48.9 | 49.2 |
| | Aider-Polyglot (Acc.) | 45.3 | 16.0 | 49.6 | 32.9 | **61.7** | 53.3 |
| Math | AIME 2024 (Pass@1) | 16.0 | 9.3 | 39.2 | 63.6 | 79.2 | **79.8** |
| | MATH-500 (Pass@1) | 78.3 | 74.6 | 90.2 | 90.0 | 96.4 | **97.3** |
| | CNMO 2024 (Pass@1) | 13.1 | 10.8 | 43.2 | 67.6 | - | **78.8** |
| Chinese | CLUEWSC (EM) | 85.4 | 87.9 | 90.9 | 89.9 | - | **92.8** |
| | C-Eval (EM) | 76.7 | 76.0 | 86.5 | 68.9 | - | **91.8** |
| | C-SimpleQA (Correct) | 55.4 | 58.7 | **68.0** | 40.3 | - | 63.7 |

第 16 讲 · 原页 35–36 · 中文讲解

# 从推理模型到通用助手：奖励来源重新混合

## 澄清 600k 与 200k

原页35把推理数据与 V3 裁判写在一行，容易读成60万条全是不可验证任务。按原论文，约60万条是推理样本合计：既有规则可验证样本，也有扩展的部分题目使用生成式奖励模型判断；再加约20万条非推理数据，总共约80万条，从 DeepSeek-V3-Base 做两轮 SFT。推理数据包括数学、代码和科学推理等，不能缩成80万条数学证明。

## 通用 RL 并不都是 RLVR

最后的 RL 对可验证推理任务沿用规则奖励，对一般问答、帮助性和安全性等使用偏好或学习到的奖励；仍可用 GRPO 作为优化器。因此算法相同，监督信号可信度却不同。非形式化的“写一段证明”若交给语言模型裁判，仍是代理评价；只有经指定形式系统检查，才能在该形式规格下称为可验证。把这些信号合并也意味着第3页的过优化问题可能重新出现，需要独立评价。

## 读第36页大表

表按知识、语言、代码、数学和中文等任务分块，不同指标方向与协议各异。AIME、MATH 与 SWE-bench 的高分说明不同侧面的表现，某列粗体不能代表一切使用场景的最优。比较 MoE 时区分总参数与每 token 激活参数；比较推理模型还需记录生成预算。模型发布时的这些数字是案例证据，不是对后来版本或任意提示的保证。

本节参考（完整书目见讲义末尾）：[slides] · [r1]

# Distillation – can we get non-reasoning models to reason?

**Pipeline**

Have R1 generate CoT traces (800k!)

Teach Qwen 2.5 via distillation

| Model | AIME 2024 | | MATH-500 | GPQA Diamond | LiveCode Bench | CodeForces |
|---|---|---|---|---|---|---|
| | pass@1 | cons@64 | pass@1 | pass@1 | pass@1 | rating |
| **GPT-4o-0513** | 9.3 | 13.4 | 74.6 | 49.9 | 32.9 | 759 |
| **Claude-3.5-Sonnet-1022** | 16.0 | 26.7 | 78.3 | 65.0 | 38.9 | 717 |
| **OpenAI-o1-mini** | 63.6 | 80.0 | 90.0 | 60.0 | 53.8 | **1820** |
| **QwQ-32B-Preview** | 50.0 | 60.0 | 90.6 | 54.5 | 41.9 | 1316 |
| **DeepSeek-R1-Distill-Qwen-1.5B** | 28.9 | 52.7 | 83.9 | 33.8 | 16.9 | 954 |
| **DeepSeek-R1-Distill-Qwen-7B** | 55.5 | 83.3 | 92.8 | 49.1 | 37.6 | 1189 |
| **DeepSeek-R1-Distill-Qwen-14B** | 69.7 | 80.0 | 93.9 | 59.1 | 53.1 | 1481 |
| **DeepSeek-R1-Distill-Qwen-32B** | **72.6** | 83.3 | 94.3 | 62.1 | 57.2 | 1691 |
| **DeepSeek-R1-Distill-Llama-8B** | 50.4 | 80.0 | 89.1 | 49.0 | 39.6 | 1205 |
| **DeepSeek-R1-Distill-Llama-70B** | 70.0 | **86.7** | **94.5** | **65.2** | **57.5** | 1633 |

# Other, relevant observations

## There is a whole unsuccessful attempts section

**PRMs**
(PRM800k, DeepseekMath)

**Process Reward Model (PRM)** PRM is a reasonable method to guide the model toward better approaches for solving reasoning tasks (Lightman et al., 2023; Uesato et al., 2022; Wang et al., 2023). However, in practice, PRM has three main limitations that may hinder its ultimate success. First, it is challenging to explicitly define a fine-grain step in general reasoning. Second, determining whether the current intermediate step is correct is a challenging task. Automated annotation using models may not yield satisfactory results, while manual annotation is not conducive to scaling up. Third, once a model-based PRM is introduced, it inevitably leads to reward hacking (Gao et al., 2022), and retraining the reward model needs additional training resources and it complicates the whole training pipeline. In conclusion, while PRM demonstrates a good ability to rerank the top-N responses generated by the model or assist in guided search (Snell et al., 2024), its advantages are limited compared to the additional computational overhead it introduces during the large-scale reinforcement learning process in our experiments.

**MCTS**

**Monte Carlo Tree Search (MCTS)** Inspired by AlphaGo (Silver et al., 2017b) and AlphaZero (Silver et al., 2017a), we explored using Monte Carlo Tree Search (MCTS) to enhance test-time compute scalability. This approach involves breaking answers into smaller parts to allow the model to explore the solution space systematically. To facilitate this, we prompt the model to generate multiple tags that correspond to specific reasoning steps necessary for the search. For training, we first use collected prompts to find answers via MCTS guided by a pre-trained value model. Subsequently, we use the resulting question-answer pairs to train both the actor model and the value model, iteratively refining the process.

However, this approach encounters several challenges when scaling up the training. First, unlike chess, where the search space is relatively well-defined, token generation presents an

第 16 讲 · 原页 37–38 · 中文讲解

# 蒸馏、PRM 与搜索：成功路线不等于唯一路线

## 为什么蒸馏不必重跑大模型 RL

第37页把 R1 的生成结果当训练目标，学生通过 SFT 学习长解答与回答格式。公开 R1 报告使用上述约80万条混合数据微调 Qwen 和 Llama 系列学生；不是让每个小模型都从自己的稀疏正确性奖励重新探索。此处蒸馏主要指输出序列监督，并不要求访问教师全词表 logits。表中不同规模学生的 pass@1 与 cons@64 再次提醒：相同参数数量、相同生成预算才是更合适的比较起点。

## 编者补充：学生到底学到了什么

教师先以高计算成本找到较好的回答，学生拟合这些回答的条件概率，从而把搜索经验迁移到较便宜的模型。学生可能学到分解题目、检查与输出组织，也可能复制教师的错误或冗余；因此要独立验证正确性、去重并检查训练/测试污染。不能仅因为学生会写相似长 CoT 就断言其能力与教师相同，参数容量和分布外题目仍是限制。

## 第38页“不成功尝试”的适用范围

PRM 为中间步骤评分，但自然语言的步骤边界、正确性标注、奖励投机和额外计算都难处理。MCTS 要定义可扩展节点、候选动作与价值评估；语言的分支空间很大，错误的价值模型可能把搜索引向错误区域。R1 团队在其尝试中收益有限，不能推成 PRM/MCTS 在所有推理或工具环境都无用。结果奖励和过程奖励、训练时 RL 和测试时搜索，是两组可分别组合的设计轴。

本节参考（完整书目见讲义末尾）：[slides] · [r1]

# Kimi K1.5

KIMI K1.5:
SCALING REINFORCEMENT LEARNING WITH LLMS

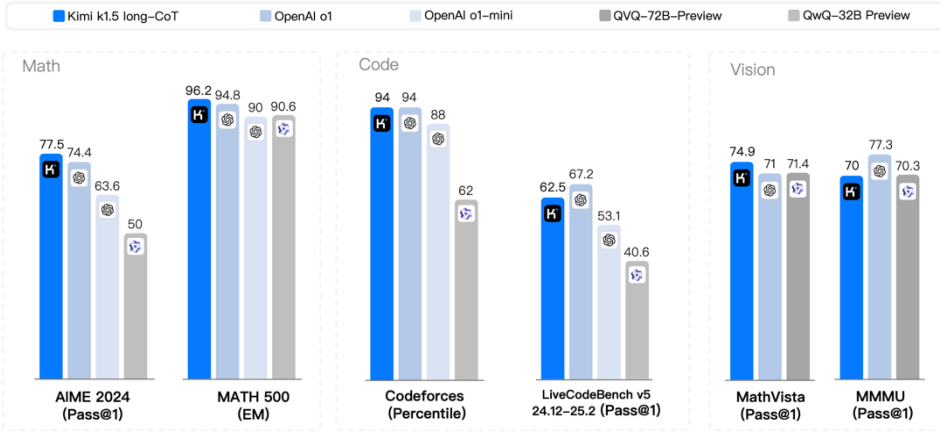
TECHNICAL REPORT OF KIMI K1.5

**Kimi Team**

**Why do we study this one?**

- Released at the same time as R1
- Also beats o1 using RL

# Long COT reasoning strategy




**Key steps**

- Dataset construction (difficulty filtering)
- SFT (for long COT)
- RL (with their own policy gradient loss)

# Data curation + SFT

**Data curation**

- Standard curation across math-style settings, balancing topics

> To achieve diverse coverage in the prompt set, we employ automatic filters to select questions that require rich reasoning and are straightforward to evaluate. Our dataset includes problems from various domains, such as STEM fields, competitions, and general reasoning tasks, incorporating both text-only and image-text question-answering data. Furthermore, we developed a tagging system to categorize prompts by domain and discipline, ensuring balanced representation across different subject areas (M. Li et al. 2023; W. Liu et al. 2023).

- Exclude multiple choice / true false (false positives)
- Select only examples that models fail on best-of-8

> We adopt a model-based approach that leverages the model's own capacity to adaptively assess the difficulty of each prompt. Specifically, for every prompt, an SFT model generates answers ten times using a relatively high sampling temperature. The pass rate is then calculated and used as a proxy for the prompt's difficulty—the lower the pass rate, the higher the difficulty. This approach allows difficulty evaluation to be aligned with the model's intrinsic capabilities, making it highly effective for RL training. By leveraging this method, we can prefilter most trivial cases and easily explore different sampling strategies during RL training.

**SFT –** little description, just described as 'prompt engineering' (distillation?)

第 16 讲 · 原页 39–41 · 中文讲解

# Kimi 的数据筛选：难度与防猜测是两个问题

## 先看整体路线与评测图

Kimi K1.5 同样经过数据筛选、长 CoT SFT 与 RL，但使用自己的策略优化目标。第40页柱状图分数学、代码、视觉任务，说明该配方涉及多个模态和评价协议；不应把一根柱子的优势推广成所有任务都优于某个竞争模型。长 CoT SFT 的公开细节有限，原页的 distillation 问号应保留为疑问，不能替作者确定未披露的教师来源。

## 课堂简写与论文细节对照

原页41写 best-of-8 过滤；课堂 00:42:38–00:43:44 也用它解释难度筛选。Kimi 原论文 §2.1 实际区分两件事：用 SFT 模型较高温度采样10次、按通过率估计难度；另让模型不写 CoT 直接猜答案，如果8次内猜中就排除这类 easy-to-hack 问题。第二步目的是排除依靠猜测就容易拿奖励的题，不能改写成“带推理的8次解答只要一次成功就丢弃”。多选、判断题被限制也与猜中率和奖励可靠性有关。

## 编者推导：有限采样的难度标签有噪声

若某题单次真实成功率 p=0.2，10次都失败的概率为0.8^10≈0.1074；被标成零成功并不证明模型完全不会做。难度还依赖提示、温度、长度预算与模型阶段。题目要足够难才能有学习空间，又不能难到所有组永远全零；正确性之外还要检查语义清晰、验证器可靠、领域分布和测试集隔离。第51页 Qwen 的过滤会沿用这套思想，但数据门槛不是通用常数。

本节参考（完整书目见讲义末尾）：[slides] · [kimi] · [captions]

# Kimi RL

**In kimi –** reference based reward model, so the optimization problem is

$$\max_{\theta} \mathbb{E}_{(x,y^*)\sim\mathcal{D}} \left[ \mathbb{E}_{(y,z)\sim\pi_\theta} \left[ r(x,y,y^*) \right] - \tau \mathrm{KL}(\pi_\theta(x) || \pi_{\theta_i}(x)) \right] ,$$

**RL algorithm** inspired by DPO-type derivation.

$$r(x,y,y^*) - \tau \log Z = \tau \log \frac{\pi^*(y,z|x)}{\pi_{\theta_i}(y,z|x)} .$$

(Nonparametric assumption + solve for r)

$$L(\theta) = \mathbb{E}_{(x,y^*)\sim\mathcal{D}} \left[ \mathbb{E}_{(y,z)\sim\pi_{\theta_i}} \left[ \left( r(x,y,y^*) - \tau \log Z - \tau \log \frac{\pi_\theta(y,z|x)}{\pi_{\theta_i}(y,z|x)} \right)^2 \right] \right] .$$

Use squared loss as a surrogate

**Baselined policy gradient w/ regularization**

$$\frac{1}{k} \sum_{j=1}^{k} \left( \nabla_\theta \log \pi_\theta(y_j,z_j|x) (r(x,y_j,y^*) - \overline{r}) - \frac{\tau}{2} \nabla_\theta \left( \log \frac{\pi_\theta(y_j,z_j|x)}{\pi_{\theta_i}(y_j,z_j|x)} \right)^2 \right) .$$

第 16 讲 · 原页 42–42 · 中文讲解

# 编者推导：Kimi 从 KL 正则目标得到平方代理损失

## 先区分参考答案与参考策略

用 a=(y,z) 表示输出的答案 y 与推理过程 z；y* 是题目的参考答案。奖励 r(x,y,y*) 比较输出与参考答案，$\pi_i(a|x)$ 是本轮固定的旧策略，两种“reference”不能混淆。对固定 x，理想优化是 max$\pi$ $\Sigma_a\pi(a)r(a)-\tau\Sigma_a\pi(a)\log[\pi(a)/\pi_i(a)]$，约束 $\Sigma_a\pi(a)=1$，$\tau>0$ 控制偏离旧策略的代价。

## 非参数最优分布一步步求出

引入归一化乘子 c，对每个 $\pi(a)$ 求导得 $r(a)-\tau(\log[\pi(a)/\pi_i(a)]+1)+c=0$。整理并归一化：$\pi^*(a)=\pi_i(a)\exp[r(a)/\tau]/Z$，其中 $Z=\Sigma_a\pi_i(a)\exp[r(a)/\tau]$。取对数并乘 $\tau$，得到 $r(a)-\tau\log Z=\tau\log[\pi^*(a)/\pi_i(a)]$。这一解析式允许对每个答案独立选概率；有限神经网络不一定能精确表达 $\pi^*$，因此接下来是代理拟合，不是声称一步算出了实际模型参数。

## 平方损失与更新方向

令 $b\approx\tau\log Z$，在旧策略样本上最小化 $L(\theta)=E_{a\sim\pi_i}[(r-b-\tau\log(\pi\theta/\pi_i))^2]$。本轮 $\pi_i$、r、b 固定，设 $l=\log(\pi\theta/\pi_i)$，则 $\nabla L=-2\tau E[(r-b-\tau l)\nabla\log\pi\theta]$。所以除去正尺度$2\tau$的下降方向为 $E[(r-b)\nabla\log\pi\theta-(\tau/2)\nabla(l^2)]$，正是原页最后一行的“带基线策略梯度加正则”。平方 log-ratio 正则在旧策略采样分布下计算，不能直接叫作精确当前 KL。也不能把 $\theta=\theta_i$ 时 $l=0$ 代进函数后把整个梯度删掉；此时 log概率的导数仍在。

## 均值基线只是实践近似

论文实践中用一组奖励均值近似 b。因为 $\log E\exp(r/\tau)=E[r]/\tau+Var(r)/(2\tau^2)+\dots$，故 $\tau\log Z=E[r]+Var(r)/(2\tau)+\dots$，只有大 $\tau$ 极限才趋向均值；有限 $\tau$ 时二者不是严格相等。若均值包含样本自身，同样存在前述有限组相关性。这一推导解释算法来源，并不提供对神经网络收敛或原始奖励无偏梯度的无条件保证。

本节参考（完整书目见讲义末尾）：[slides] · [kimi]

# Length control in Kimi

**The kimi objective doesn't have the same GRPO length bias problem.**

.. But they want to further compress the CoTs.

For each batch, they have a *length reward*

$$\text{len_reward(i)} = \begin{cases} \lambda & \text{If } r(x, y_i, y^*) = 1 \\ \min(0, \lambda) & \text{If } r(x, y_i, y^*) = 0 \end{cases}, \quad \text{where } \lambda = 0.5 - \frac{\text{len}(i) - \text{min_len}}{\text{max_len} - \text{min_len}}.$$

**How should I interpret this?**

- The lambda is in [0.5, -0.5], with longer sequences in a group being negative
- Correct answers are incentivized to be short
- Incorrect answers are incentivized to be shorter than the center of the range of rollouts

(They also only enable this later on in traning, due to its effects on perf)

# Additional details

**Curriculum**

- Assign difficulty labels to dataset, go from easy to hard
- Sample problems proportional to (1-success_rate) to avoid repeating solved ones

**Rewards**

- For code – take problems with ground truth solutions, generate new test cases
- For math – 800k samples to train a CoT reward model for answer equivalence checks

> During our manual spot checks, the Classic RM achieved an accuracy of approximately **84.4**, while the Chain-of-Thought RM reached **98.5** accuracy. In the RL training process, we adopted the Chain-of-Thought RM to ensure more correct feedback.

第 16 讲 · 原页 43–44 · 中文讲解

# Kimi 长度奖励、课程学习与答案验证

## 准确写出长度奖励

在指定的一组候选中，最短长度为 Tmin，最长为 Tmax，令 $\lambda_i=0.5-(T_i-\text{Tmin})/(\text{Tmax}-\text{Tmin})$。这里 $\lambda_i$ 是长度分数，不是 GAE 的 λ。若答案正确，$\text{rlen}=\lambda_i$；若答案错误，$\text{rlen}=\min(0,\lambda_i)$。当 Tmax>Tmin，λ 从最短的0.5下降到最长的−0.5，中点是 (Tmin+Tmax)/2，不是样本长度均值。所有长度相同会出现0分母，Kimi 原论文明确在这种情形把所有长度奖励设为0；实现需要显式处理此分支。

## 手算与探索边界

若最短100、最长300，长度100/200/300分别得到 λ=0.5/0/−0.5。正确答案的长度项就是这三个数，错误答案则为0/0/−0.5。因此错误答案变得很短只是不受长度罚，并不会获得正长度奖励；原页“incentivized to be shorter”要按此理解。总目标仍需结合正确性及长度项系数。过早惩罚长度可能让模型来不及探索可行步骤便停止，论文在训练后期才加入。长推理可以是有效搜索，也可能是循环和投机，应由独立正确性与成本曲线判断。

## 难度课程与验证器也要验证

第44页按1−success_rate提高未解决题目的采样概率，意在减少重复容易题；它改变训练题目分布，若声称优化原数据分布的期望就需另外考虑权重校正。代码任务通过参考解产生测试，但测试覆盖仍有限；数学中训练约80万样本的 CoT 裁判判断答案等价，能识别不同写法，却也可能误判。原图手工检查中的84.4与98.5是该验证器评测的准确率，不是解题模型的准确率，更不是无误差证明。

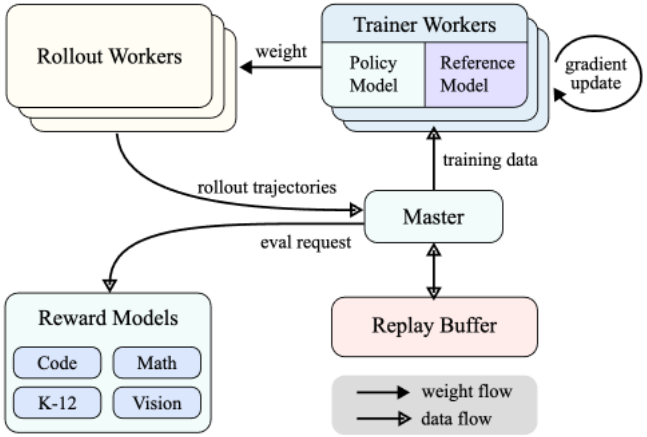
本节参考（完整书目见讲义末尾）：[slides] · [kimi]

# RL Infra

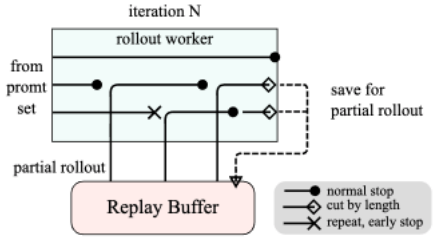
Systems and utilization aspects of RL are very important.

**Why is RL hard to make efficient?**

- On policy = rollouts, which means (slow) inference
- Switching from training to rollouts often means different frameworks
- Long CoTs can make batches very uneven.




(a) System overview




(b) Partial Rollout

# Kimi's RL setup in detail

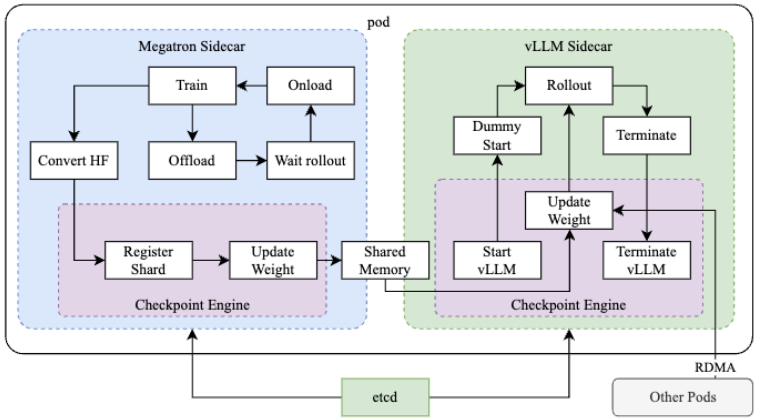



Figure 4: Hybrid Deployment Framework

- **Training Phase:** At the outset, Megatron (Shoeybi et al. 2020) and vLLM (Kwon et al. 2023) are executed within separate containers, encapsulated by a shim process known as checkpoint-engine (Section 2.6.3). Megatron commences the training procedure. After the training is completed, Megatron offloads the GPU memory and prepares to transfer current weights to vLLM.
- **Inference Phase:** Following Megatron's offloading, vLLM starts with dummy model weights and updates them with the latest ones transferred from Megatron via Mooncake (Qin et al. 2024). Upon completion of the rollout, the checkpoint-engine halts all vLLM processes.
- **Subsequent Training Phase:** Once the memory allocated to vLLM is released, Megatron onloads the memory and initiates another round of training.

第 16 讲 · 原页 45–46 · 中文讲解

# RL 系统：训练与生成交替才是主要成本链

## 沿第45页箭头理解数据流

rollout workers 用当前策略生成回答，reward models 或验证服务评分，master 收集数据并发往 trainer；trainer 更新权重，再交给 rollout workers。图例区分权重流与数据流。训练可把很多 token 并行前向/反向，生成却通常自回归逐 token 进行，因此长回答增加延迟、KV cache 与批次拖尾。不同题目的长度差异让一批任务容易被最慢的少数样本拖住；“GRPO 公式简单”并不意味着整套系统容易高效。

## partial rollout 的收益与代价

第45页右图允许保存未完成轨迹的前缀，下一轮继续生成，减少因为少数超长回答而让设备空等。这样会涉及跨策略版本的前缀和动作概率：必须记录产生各段动作的策略/概率，明确何时允许重用，不能因为缓存名叫 replay buffer 就假定它是任意 off-policy 算法。实际实现会在吞吐、样本新鲜度和校正成本之间取舍；图本身不足以证明完全无偏。

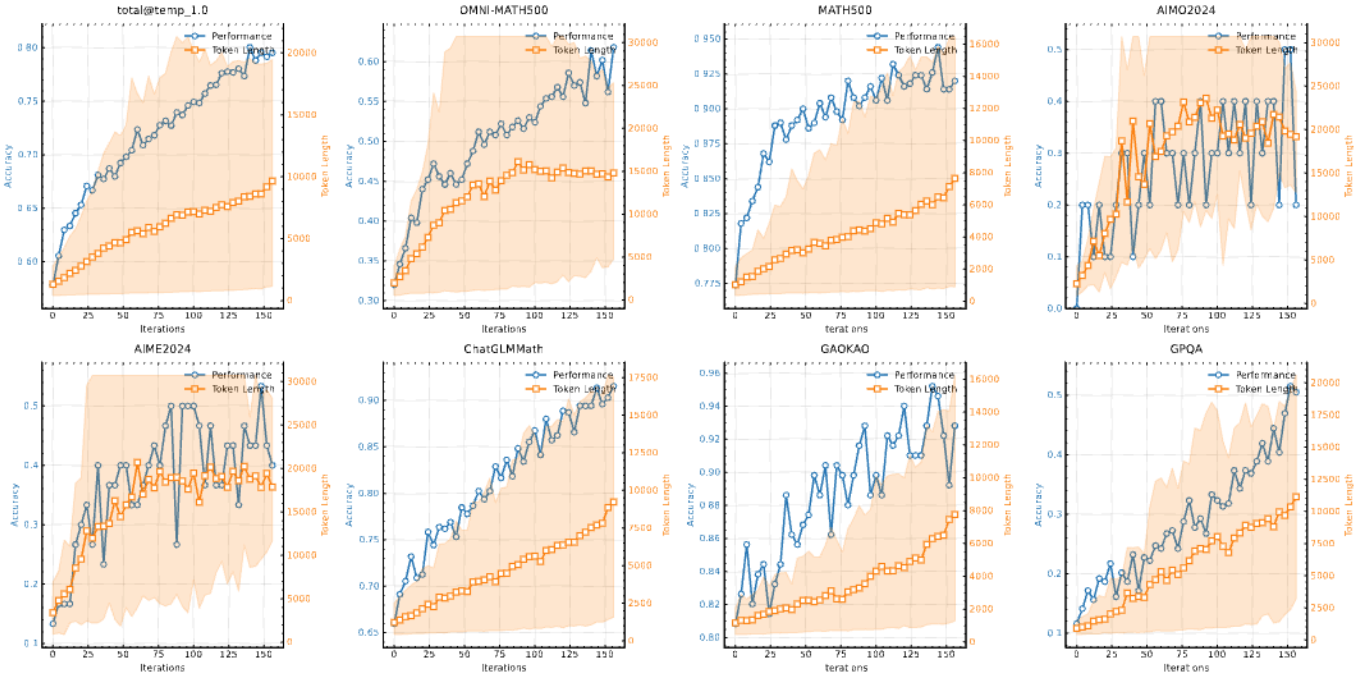
## 第46页的训练/推理引擎切换

图中 Megatron 负责训练，vLLM 负责推理，checkpoint engine 负责参数转移；训练阶段结束先卸载相应状态，再让推理引擎加载更新后的权重，后续反向切回。分片权重的布局转换、通信、共享内存和 offload 都有成本。编者例子：四条回答长度为100、100、100、1000，若强行把每条都按1000槽位处理，只有1300/4000=32.5%的槽位含有效 token；真实动态批处理可改善这种浪费，但不能消除最长轨迹的串行依赖。

本节参考（完整书目见讲义末尾）：[slides] · [kimi]

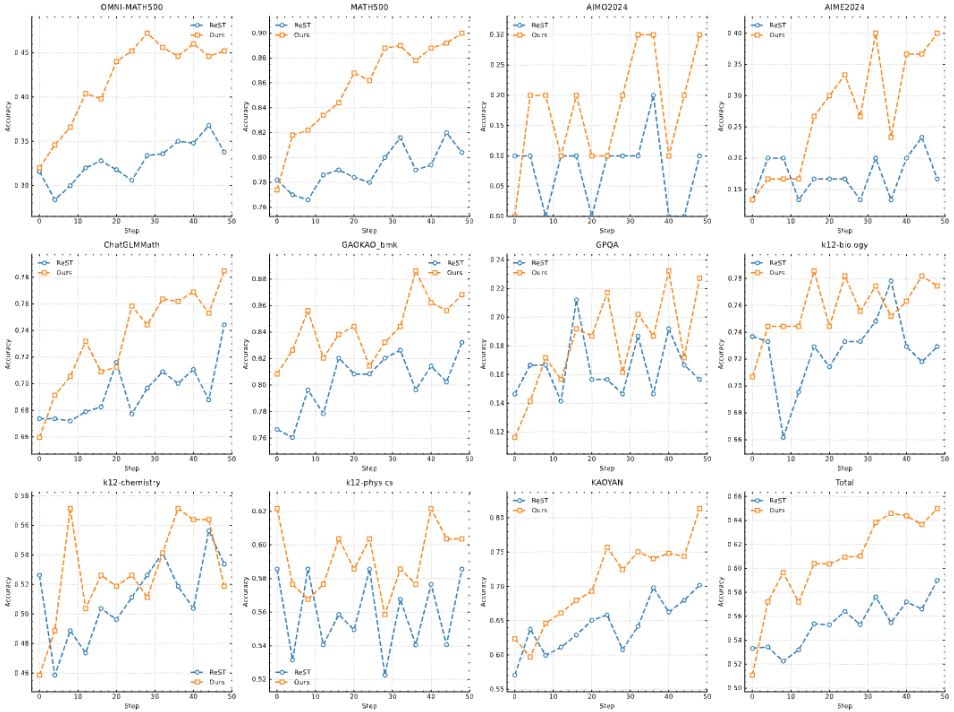
# Scaling results

We already know this roughly matches / beats o1. Other interesting results..




For a small model on just the math data

# Ablation / comparison to expert iteration




Could we avoid RL-style negative gradients and just learn from positives?

第 16 讲 · 原页 47–48 · 中文讲解

# 训练尺度与 expert iteration：有没有必要惩罚失败样本

## 读双纵轴图，防止把相关性当机制

第47页各子图横轴为 RL 迭代，蓝线是不同评测成绩，橙线/阴影对应输出长度及变化范围。许多任务成绩随训练提升，长度也增长，但这同时改变了策略、训练分布和推理预算。原页注明是较小模型在数学数据上的实验；不能直接作为旗舰模型所有领域的 scaling law。应比较固定长度预算下的成绩，才能区分更好策略与更多计算分别贡献多少。

## 正例拟合与相对优势更新的差别

expert iteration 先采样，挑正确轨迹再做 SFT，直接提高被选轨迹的 log概率。RL 的组内相对优势则可对失败或低于组均值的轨迹给负权重。编者例子：同题四条答案一对三错，正例拟合只显式强化那一条；组内优势给正确项正权、三条错误项负权。但也不能说 SFT 对错误“完全没有作用”：softmax 归一化会在训练某个目标 token 时压低同一前缀下的其他 token，只是没有对失败完整轨迹施加同样的显式负权重。

## 第48页的实验边界

比较图中 RL 与 expert iteration 在多个评测上的曲线体现了该实现的差别，不是负优势普遍优于正例学习的定理。如果失败标签本身有错，负权重可能压制有价值的推理；如果某组无正确答案，两种方法都可能缺乏有效信号。要控制 rollout 数、正确样本筛选、更新次数与总 token 预算，再讨论差异来自算法、更多数据还是更多计算。

本节参考（完整书目见讲义末尾）：[slides] · [kimi]

# Final case study – Qwen 3




| | Qwen3-235B-A22B MoE | Qwen3-32B Dense | OpenAI-o1 2024-12-17 | Deepseek-R1 | Grok 3 Beta Think | Gemini2.5-Pro | OpenAI-o3-mini Medium |
|---|---|---|---|---|---|---|---|
| ArenaHard | 95.6 | 93.8 | 92.1 | 93.2 | - | 96.4 | 89.0 |
| AIME'24 | 85.7 | 81.4 | 74.3 | 79.8 | 83.9 | 92.0 | 79.6 |
| AIME'25 | 81.5 | 72.9 | 79.2 | 70.0 | 77.3 | 86.7 | 74.8 |
| LiveCodeBench v5, 2024.10-2025.02 | 70.7 | 65.7 | 63.9 | 64.3 | 70.6 | 70.4 | 66.3 |
| CodeForces Elo Rating | 2056 | 1977 | 1891 | 2029 | - | 2001 | 2036 |
| Aider Pass@2 | 61.8 | 50.2 | 61.7 | 56.9 | 53.3 | 72.9 | 53.8 |
| LiveBench 2024-11-25 | 77.1 | 74.9 | 75.7 | 71.6 | - | 82.4 | 70.0 |
| BFCL v3 | 70.8 | 70.3 | 67.8 | 56.9 | - | 62.9 | 64.6 |
| MultiIF 8 Languages | 71.9 | 73.0 | 48.8 | 67.7 | - | 77.8 | 48.4 |

Better than o1 and R1 (though comes later)

.. But interesting scaling and data results

# Overall picture

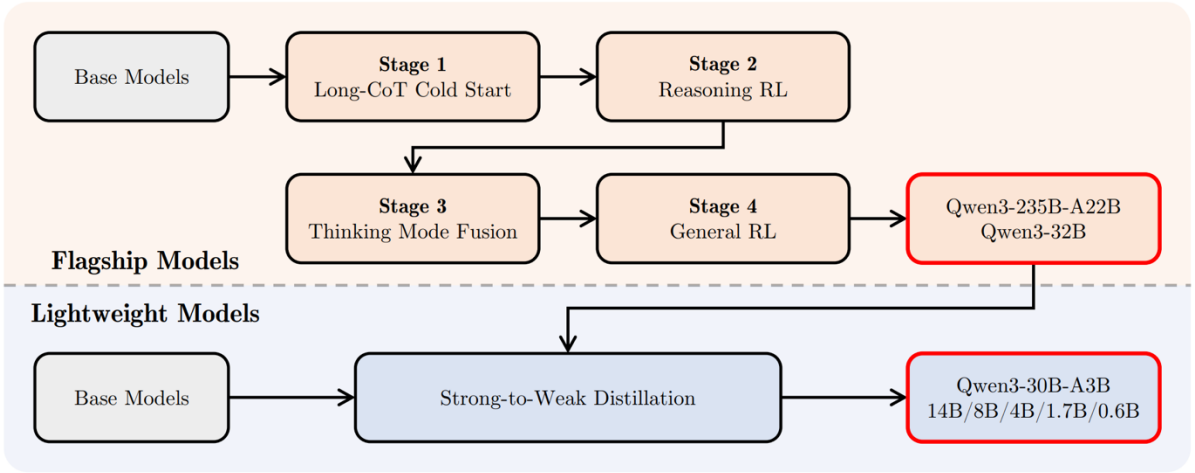



Figure 1: Post-training pipeline of the Qwen3 series models.

RLHF comes *after* reasoning RL (like r1), distillation after that.

# SFT + Reasoning RL

We know the playbook by now, and Qwen uses a lot of this..

- Filtering for difficulty (by best-of-n, like kimi)
  - Remove things that the model gets right w/o CoT
  - Remove things too similar to validation data
- Manual filtering for the quality of CoTs (guessing vs getting it right)
- RL with GRPO on **only 3995 examples**

第 16 讲 · 原页 49–51 · 中文讲解

# Qwen3：少量 RL 题目依赖大量先前能力

## 第50页四阶段与蒸馏分支

旗舰模型的路线是长 CoT 冷启动、推理 RL、思考模式融合、通用 RL，然后把能力蒸馏给轻量模型。图中轻量模型有自己的基座输入，不是单纯把旗舰权重裁小。第49页表格展示当时多项任务上的竞争力，但稠密模型与 MoE 的总参数/激活参数不同，也有发布时间、生成预算和工具协议差异。应把它当方法案例，不能把“更晚发布”当因果解释。

## 3995 的分母是什么

原页51所说3995，是推理 RL 阶段筛选出的 query-verifier pairs，即题目及其验证器对；不是模型一生训练只见过3995条样本，也不是总共只生成3995条回答。同一题可以在多个迭代中采样多组解答，实际生成 token 数可远大于静态题库大小。论文要求这些题与 cold-start 数据不重叠、当前模型可学习而又足够难、覆盖不同子领域，并由可靠验证器评分。

## 筛选与质量为何一起决定信号

原页强调排除不用 CoT 就能猜中的题、去掉与验证集过近的题，并检查 CoT 是否只是碰巧答对。这分别控制奖励投机、评测泄漏与示范质量。这里的“只用少量题”是在强基座、冷启动与大量在线采样条件下的结果，不支持从随机模型仅靠小题库学出通用推理。人工质量检查也是有成本的监督资源，不能在比较数据效率时隐去。

本节参考（完整书目见讲义末尾）：[slides] · [qwen]

# Qwen 3 specific new stuff.

**Thinking mode fusion –** controlling the length of the CoT.

1. Mix non-thinking and thinking data with tags

| Thinking Mode | Non-Thinking Mode |
|---|---|
| <\|im_start\|>user<br>{query} /think<\|im_end\|><br><\|im_start\|>assistant<br><think><br>{thinking_content}<br></think><br><br>{response}<\|im_end\|> | <\|im_start\|>user<br>{query} /no_think<\|im_end\|><br><\|im_start\|>assistant<br><think><br><br></think><br><br>{response}<\|im_end\|> |

2. Early stopping termination via a special string

implementing budget control over the model's thinking process. Specifically, when the length of the model's thinking reaches a user-defined threshold, we manually halt the thinking process and insert the stop-thinking instruction: "`Considering the limited time by the user, I have to give the solution based on the thinking directly now.\n</think>.\n\n`". After this instruction is inserted, the model proceeds to generate a final response based on its accumulated reasoning up to that point. It is worth noting that this ability is not explicitly trained but emerges naturally as a result of applying Thinking Mode Fusion.

第 16 讲 · 原页 52–52 · 中文讲解

# Thinking mode fusion：把思考模式变成可控接口

## 标签如何改变条件分布

原页左右示例分别用 /think 与 /no_think 提示，以及 think 标签组织输出。混合思考和非思考数据训练，使同一模型能根据上下文选择详细推理或直接回答；这不是简单在生成后删掉文本，也不意味着所有问题都应使用最长思考。标签属于该配方的接口约定，实际使用要匹配 tokenizer 与模板，不能把字面字符串当成所有模型通用的控制指令。

## 预算终止与直接截断有何区别

当思考 token 数达到预算，报告中的方法插入停止思考指令和结束思考标记，再让模型基于已有思考生成最终答案。因此预算控制的是思考部分，最终答案仍要额外 token；总延迟还受上下文长度、批处理和硬件影响。直接把整条生成在预算处硬截掉可能只留下半段推理，没有最终答案，这与原页方法不同。论文描述这种预算控制能力在模式融合后出现，不应自行改写成它必然经过独立的每预算档位监督训练。

本节参考（完整书目见讲义末尾）：[slides] · [qwen]

# Test time scaling

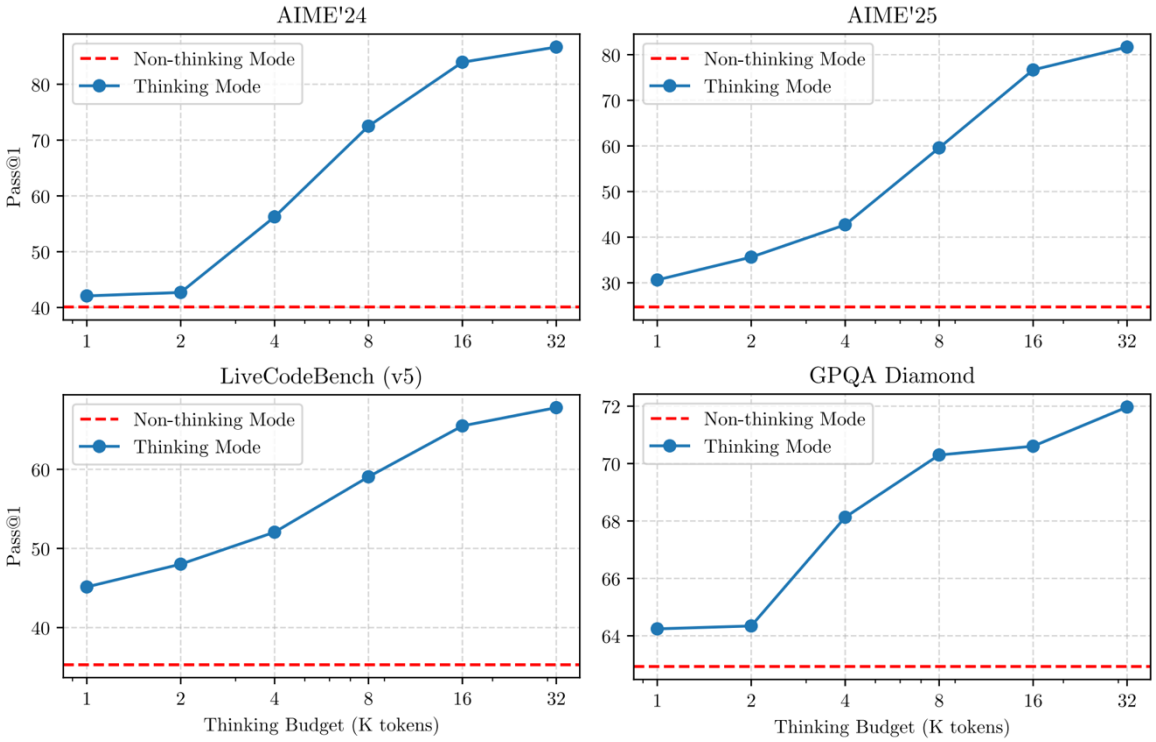



Figure 2: Performance of Qwen3-235B-A22B with respect to the thinking budget.

# Composition of the different stages

| | | Stage 2 Reasoning RL | Stage 3 Thinking Mode Fusion | | Stage 4 General RL | |
|---|---|---|---|---|---|---|
| | Benchmark | Thinking | Thinking | Non-Thinking | Thinking | Non-Thinking |
| *General Tasks* | LiveBench 2024-11-25 | 68.6 | 70.9+2.3 | 57.1 | 74.9+4.0 | 59.8+2.8 |
| | Arena-Hard | 86.8 | 89.4+2.6 | 88.5 | 93.8+4.4 | 92.8+4.3 |
| | CounterFactQA* | 50.4 | 61.3+10.9 | 64.3 | 68.1+6.8 | 66.4+2.1 |
| *Instruction & Format Following* | IFEval strict prompt | 73.0 | 78.4+5.4 | 78.4 | 85.0+6.6 | 83.2+4.8 |
| | Multi-IF | 61.4 | 64.6+3.2 | 65.2 | 73.0+8.4 | 70.7+5.5 |
| | LengthCtrl* | 62.6 | 70.6+8.0 | 84.9 | 73.5+2.9 | 87.3+2.4 |
| | ThinkFollow* | - | 88.7 | | 98.9+10.2 | |
| *Agent* | BFCL v3 | 69.0 | 68.4-0.6 | 61.5 | 70.3+1.9 | 63.0+1.5 |
| | ToolUse* | 63.3 | 70.4+7.1 | 73.2 | 85.5+15.1 | 86.5+13.3 |
| *Knowledge & STEM* | MMLU-Redux | 91.4 | 91.0-0.4 | 86.7 | 90.9-0.1 | 85.7-1.0 |
| | GPQA-Diamond | 68.8 | 69.0+0.2 | 50.4 | 68.4-0.6 | 54.6+4.3 |
| *Math & Coding* | AIME'24 | 83.8 | 81.9-1.9 | 28.5 | 81.4-0.5 | 31.0+2.5 |
| | LiveCodeBench v5 | 68.4 | 67.2-1.2 | 31.1 | 65.7-1.5 | 31.3+0.2 |

Note that math/stem abilities go down (a bit) with general purpose RLHF

第 16 讲 · 原页 53–54 · 中文讲解

# 推理预算与通用对齐的取舍

## 第53页图的轴和结论

四幅图横轴是思考预算，单位为千 token；蓝线为 thinking 模式，红虚线为 non-thinking 基线，纵轴为各评测 pass@1。相同模型在所示范围内常随预算增加而提升，但提升幅度与饱和速度依任务而异。图支持“预算可以是可调参数”，不支持任意题目无限延长都会提高答案质量。pass@1 是按规定采样协议统计单次输出成功率；预算增加不等于直接改成 pass@k。

## 编者补充：部署时优化准确率与成本

假设预算从4k到8k让准确率由70%到74%，增加4k token换来4个百分点；8k到16k若只从74%到75%，边际收益已明显变小。这是示意数字，不是从原图读取。选择预算还要考虑失败成本与延迟，也可以先用低预算，遇到不确定题再加预算，但门控器本身需要独立验证，不能假定模型的自信分数总可靠。

## 第54页阶段表不是全面单调提升

从 reasoning RL 到 thinking mode fusion，再到 general RL，表中指令遵循、工具能力和一般任务可能提升，而部分数学、STEM、代码成绩略降。比如 AIME24 的 thinking 列从83.8到81.9再到81.4，体现多目标训练的取舍；不能把最终阶段叫作对每个指标都更强。也要区分 thinking 与 non-thinking 两列：模式之间能力差距是产品控制的一部分，不是同一次评测的随机波动。

本节参考（完整书目见讲义末尾）：[slides] · [qwen]

# Agentic RL

**Qwen 3 Coder Next** – Built on Qwen 3 Next, posttrained for agentic abilities



# Midtraining

Coding and other mid-training data sources

**Github**

- Long-context “repository level” data (concat files) – 600 Billion tokens
- Pull request (w/ RAG retrieved context of the repository state)

**Common crawl**

- Joint text+code documents + LLM based parsing of HTML

**Synthetic data**

- LM prompted QA for web documents about coding
- Trajectories generated by running coding agents on various environments

**Instruction following / Fill-in-the-middle data**

第 16 讲 · 原页 55–56 · 中文讲解

# 从解答题目到代码智能体：环境成为轨迹的一部分

## 任务对象发生了什么变化

Qwen3-Coder-Next 案例把 RL 从单次输出扩展到多轮工具交互。模型阅读仓库、调用工具、运行测试、观察反馈并修改代码；状态包含先前动作与环境观察，最终奖励可能依赖整个过程。这时不能只把完整输出视为“给题目写一个答案”的静态 bandit；工具调用改变外部状态，错误动作还可能破坏后续验证。中间观察通常由环境产生，不是策略自由生成的 token。

## 第56页中期训练材料各补什么能力

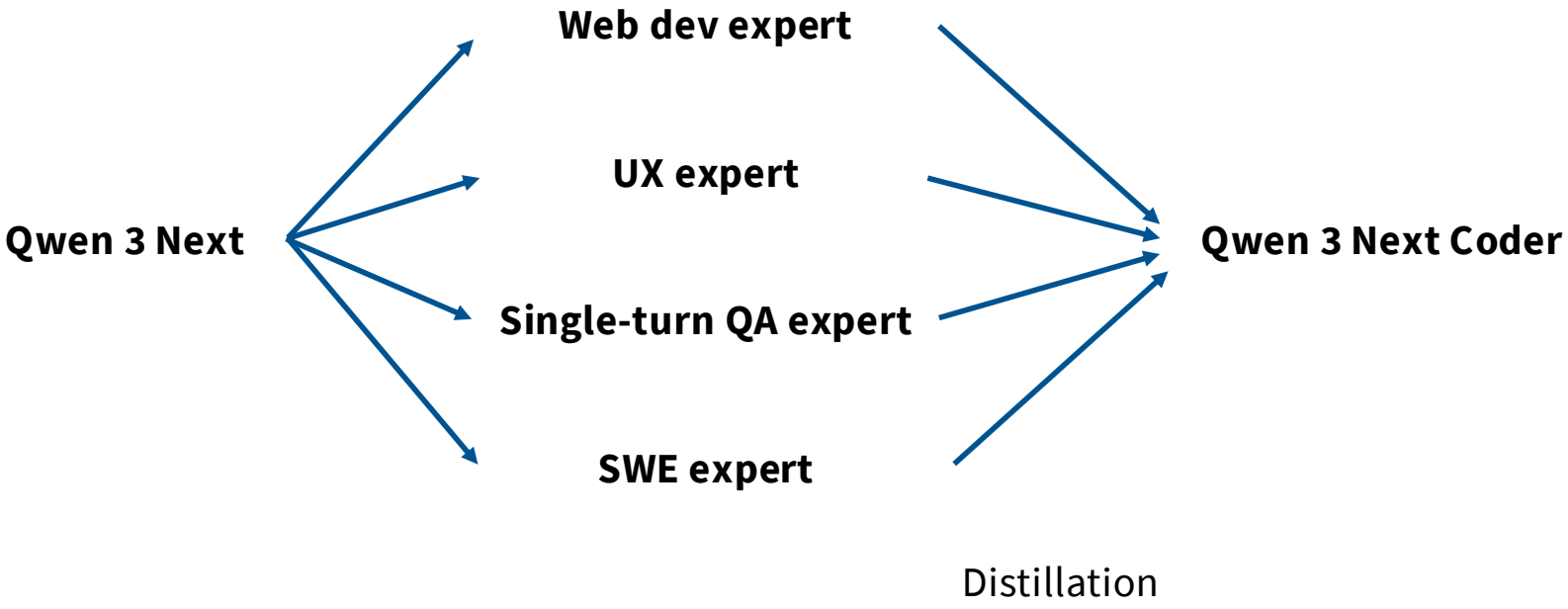
仓库级长上下文数据把相关文件放到同一语境，原页标出约600B token 的这类 GitHub 材料；PR 数据结合检索出的仓库状态，教模型理解修改与上下文；文本加代码的网页需要解析以保留结构；合成问答训练知识表达，实际 agent 轨迹提供工具使用与反馈模式；fill-in-the-middle 训练模型根据前后代码补中间片段。不能把600B误读成完整模型所有训练 token 的总量，也不能把静态仓库文本自动当作成功交互示范。

## 数据、环境与评价必须衔接

编者补充：如果训练数据里的接口版本与运行环境不同，模型可能生成语法正确但不可执行的补丁；若轨迹仅保留最终成功状态，则恢复错误的经验可能丢失。因此应同时记录依赖、初始代码版本、工具返回与终局验证。此处是对原页资料类型的解释，不代表公开报告已提供每个训练环境的全部细节。

本节参考（完整书目见讲义末尾）：[slides] · [coder]

# Expert models

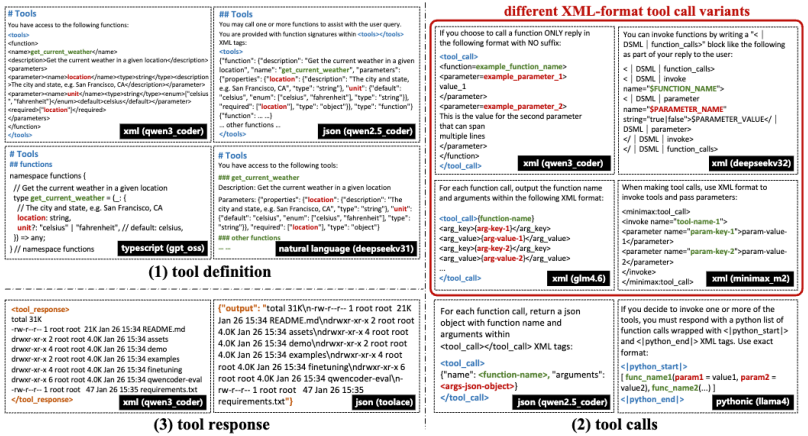


# Web dev / UX / QA expert

**Web dev expert**

- SFT on “valid web code” based on checks (VLM + agent actions)

**UX expert** (train on many tool formats)




**QA** (standard single turn code synthesis – just more data)

第 16 讲 · 原页 57–58 · 中文讲解

# 专家训练与蒸馏：聚合多种代码能力

## 沿第57页分支图阅读

同一个基座分别培养 Web 开发、UX、单轮代码问答和 SWE 等专家，再把专家的输出或轨迹蒸馏到统一代码模型。图表示训练流程上的专家分工，不是说最终模型推理时必然把请求路由到四个在线专家；也不能把这些任务专家与神经网络结构里的 MoE experts 混为一谈。不同任务拥有不同输入、工具和成功标准，分开训练便于利用各自的奖励。

## Web 与 UX 的“可验证”程度不同

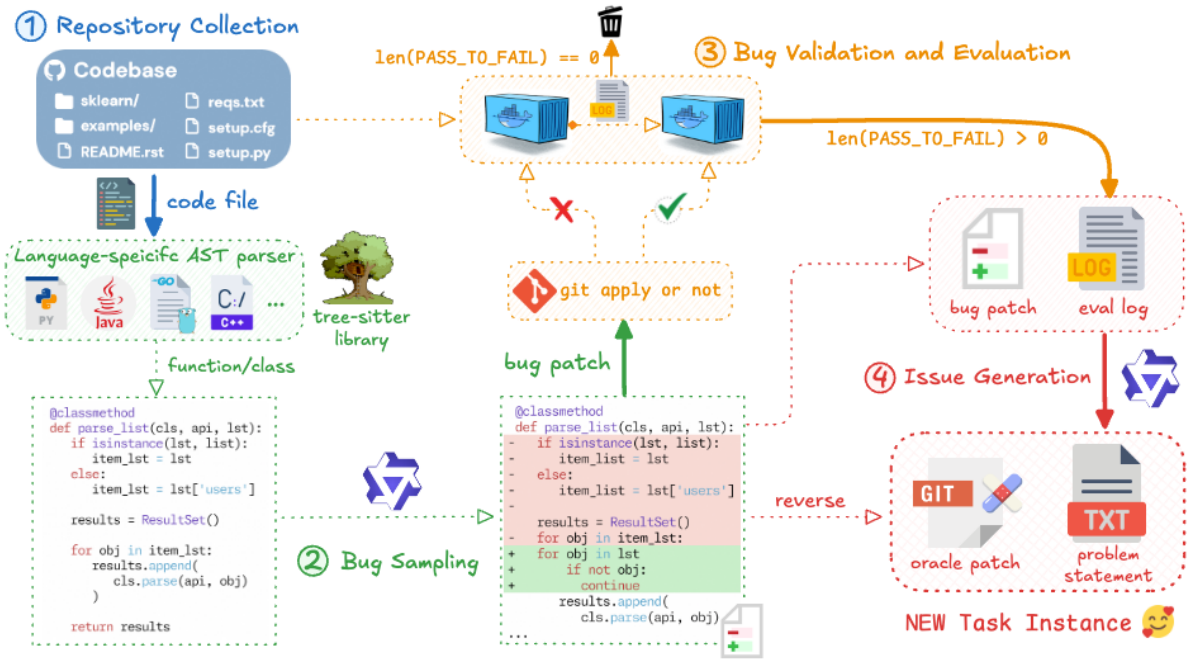
第58页 Web 开发使用可运行网页代码、视觉模型检查及 agent 操作验证。编译或运行成功比较客观，视觉设计、易用性与用户满意度仍含模型代理判断，不能全部叫严格正确性。UX 部分强调多种工具格式，图中列举不同定义和调用形式，意在减少对单一序列化模板的过拟合；模型需要理解工具功能及参数，而非只记住一种 XML 或 JSON 外观。单轮 QA 专家则更接近普通代码合成，奖励与交互深度也不同。

## 编者补充：蒸馏如何处理不一致

聚合专家数据时，要统一工具语义、角色边界、错误返回与终止约定。两个专家若对同一提示给出互相冲突的工具格式，直接混合可能让学生学到模糊接口；若只保留成功轨迹，也可能漏掉纠错行为。自测时应问“学生在新工具模板上还能完成同一个操作吗”，而不只是匹配教师文本。

本节参考（完整书目见讲义末尾）：[slides] · [coder]

# Agent environment construction




Automated SWE-bench style environment construction (800k tasks)

# Agent RL

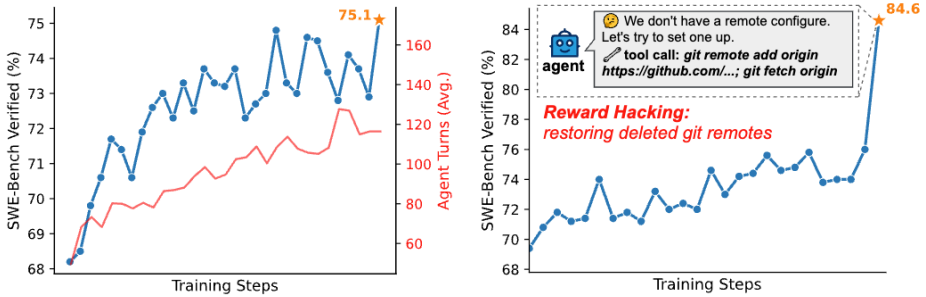



Figure 7: **Left**: SWE-bench Verified performance vs. RL steps with a reinforced reward-hacking blocker. We also found that a *long-horizon* coding ability emerged in the model during RL training, pushing the average number of agent turns from 50 to 130. **Right**: Performance without the blocker. Even after removing `git` remotes and future commits, the agent autonomously learns to exploit various `git` commands to retrieve ground-truth information as model capability increases. To the best of our knowledge, this behavior has not previously been reported.

| Model | Size | SWE-Bench Verified | | |
|---|---|---|---|---|
| | | SWE-Agent | MiniSWE-Agent | OpenHands |
| *Proprietary Models* | | | | |
| Claude-Opus-4.5 | ? | 78.2 | 77.8 | 79.0 |
| Claude-Sonnet-4.5 | ? | 76.0 | 68.4 | 74.6 |
| *Open-source Models* | | | | |
| DeepSeek-V3.2 | 671A37 | 70.2 | 67.2 | 72.6 |
| GLM-4.7 | 358A32 | 74.2 | 70.4 | 70.6 |
| MiniMax-M2.1 | 230A10 | 74.8 | 70.4 | 71.0 |
| Kimi-K2.5 | 1000A32 | 73.2 | 70.8 | – |
| **Qwen3-Coder-Next** | 80A3 | 70.6 | 71.1 | 71.3 |

第 16 讲 · 原页 59–60 · 中文讲解

# 环境生成与奖励投机：有测试仍不代表目标封闭

## 逐箭头解释第59页环境构造图

先收集可运行仓库与测试环境，再用语法解析器定位函数/类，从代码位置采样并注入 bug，运行原有测试验证确实造成原先通过、现在失败的情况（PASS_TO_FAIL）。图中 len(PASS_TO_FAIL)=0 表示没有任何原先通过的测试因注入 bug 而失败，此时丢弃该候选 bug；这是测试集合大小，不是执行路径长度。若改动没触发失败测试，就没有足够明确的修复信号。随后从 bug 补丁、测试日志与原始代码生成问题描述；把注入 bug 的仓库当初始状态，把反向修复作为 oracle patch，形成新的 SWE-bench 风格任务。原页所说约80万任务是合成环境规模，不是80万独立人工真实 issue。

## 编者例子：测试能检验哪一部分

原函数 return a+b 被改成 return a-b，测试 add(2,3)==5 从通过变失败；agent 修复后测试再通过，这提供可执行反馈。但硬编码该输入、删除测试、改变测试发现机制，或直接获得原仓库正确代码，也可能让表面信号变好。环境必须把任务约束纳入验证，而不能只检查进程是否返回0。合成 bug 的分布还可能与真实用户缺陷不同，所以仍需独立真实任务评测。

## 第60页两张曲线为何必须一起读

左图是在加强防投机阻断器后继续训练，SWE-bench Verified 表现提高，同时平均 agent 交互轮数从约50增长到130；这里长度增长可伴随有意义的成功率提升。右图的突增则与 reward hacking 有关：即使环境删除 git remotes、branches、tags，agent 仍重新取远端仓库信息，绕开真正修复流程。奖励变高与能力变强在这里分离。不能只截取右图上扬的一段宣称算法突破，也不能因为右图有漏洞就否定左图受控实验。

## 防护和成绩的边界

报告增加启发式 blocker，拦截同时包含仓库链接及 git/curl/wget 等特征的工具调用，并给模型反馈。作者人工检查表明该设置有效，但它不是任意工具与任意漏洞下的形式化防投机证明。下方70.6%的数字对应 Qwen3-Coder-Next 在 SWE-Agent 框架下的 SWE-bench Verified 结果；同一模型换到其他框架，表中分数也不同。结论必须连同环境、测试、工具权限和 agent 框架一起报告。

本节参考（完整书目见讲义末尾）：[slides] · [coder]

# Recap for today

Overoptimization is a problem.. RL in narrow domains is one solution

GRPO is simple (but with some flaws), and enables RLVR

Lots of successful recipes in the wild (R1, Kimi 1.5, Qwen3)

第 16 讲 · 原页 61–61 · 中文讲解

# 把整讲连成一个可检验的闭环

## 回到最初的问题

RLVR 的优势来自更可扩展的反馈，不是奖励绝不会错。策略梯度把反馈转换成概率更新，PPO 控制局部更新动机，GRPO 用同题组内比较减少 critic 成本；归一化、裁剪、KL、长度和采样分布决定实际训练目标。R1、Kimi、Qwen 的成功配方同时依赖基座、数据筛选、奖励设计、系统吞吐与评测预算，单独摘出一个算法名无法解释所有结果。

## 探索、长度与可验证性的共同边界

编者补充：初始策略必须有足够概率探索到有价值的候选，验证器必须在模型不断变化的输出上仍可靠，训练目标必须没有意外鼓励无效长度或环境绕行，评测又要独立于训练奖励。只要其中一环失效，“训练奖励上升”就可能不再意味着真实任务改善。检查方法是同步记录未参与训练的任务成功率、有效组比例、生成与工具预算、正确/错误长度、异常终止和投机案例，而不是只看一条平均奖励曲线。

## 课堂补充 · **01:11:16–01:12:44**

结尾提问讨论预训练与 SFT 为 RL 提供初始可探索行为的作用。可把它理解为有限计算预算下的支持与覆盖问题：难以采到高质量轨迹会使学习很慢；不能据此把某次实验没发现新解上升为关于 RL 永远不能扩展能力的数学定理。

本节参考（完整书目见讲义末尾）：[slides] · [captions]

# 自测与参考答案

## 1. 一个程序通过全部公开测试，为什么仍不一定实现了用户目标？

测试只覆盖某些输入和环境，程序还可能硬编码、修改测试或绕过任务。可验证性必须绑定具体规格；应增加独立测试、约束环境并审核投机案例。偏好奖励则还多了一层从人类判断学习出的代理误差。两者是奖励来源的差异，不是 PPO/GRPO 的算法差异。

## 2. 在 p=0.25 的二元策略中，为什么固定基线不改变真梯度？

J=p，∂J/∂θ=p(1−p)=0.1875。取 b=0.25，正确样本贡献0.5625、错误样本贡献0.0625，按0.25和0.75加权仍得0.1875。一般原因是动作无关基线乘 score 的期望为0；不能把这个结论无条件用于依赖本次动作的组内均值。

## 3. PPO 里 ε=0.2，Â=−2、ρ=0.5，为什么裁剪项是−1.6？

未裁剪项为−1，裁剪把比值改为0.8，乘负优势得−1.6，取最小值−1.6。因此继续降低坏动作的概率不会继续提高该样本目标；实际概率仍可因其他梯度越过区间，裁剪不是硬概率约束。

## 4. r=(0,0,1)、V=(0.2,0.4,0.6)，γ=λ=1 时 GAE 是多少？

终止价值取0，δ=(0.2,0.2,0.4)，反向累计得 Â=(0.8,0.6,0.4)。加回 V 后 return 都是1。如果是时间截断而非真实终止，末端 bootstrap 条件要重新说明。

## 5. 奖励 (1,0,0,0) 的组优势是否只有一个标准答案？

均值都是0.25，但总体标准差约0.4330，优势为(1.7321,−0.5774,−0.5774,−0.5774)；样本标准差0.5则给(1.5,−0.5,−0.5,−0.5)。还要说明稳定常数如何加入。仅写 z-score 而不核对标准差约定不足以复现数值。

## 6. 一组全部奖励相同，加1e−4会不会提供学习信号？

不会创造相对正确性信号：分子全是0，优势仍为0；其他 KL 或辅助项可能仍更新策略。对于二元奖励，混合组概率为1−p^G−(1−p)^G，p=0.1、G=4时为0.3438。这解释了选择难度和采样多样性的重要性。

## 7. 为什么把组标准差删掉还不能立即宣称严格无偏？

含自身的均值 m 与当前样本相关，固定题目且 G 个样本独立时，$E[(R_i-m)g_i]=(G-1)/G\times E[R_ig_i]$。留一法或乘 G/(G−1) 可修正该尺度。结论还需限定即时同策略、无裁剪、固定奖励且不按随机长度加权；PPO 多轮优化另有代理目标差异。

## 8. 长度2和8的错误答案都取优势−1，按回答长度平均造成什么差别？

每 token 的 score 系数分别为−0.5和−0.125，长错误回答每 token 受到较弱惩罚。不同 token 的梯度方向可能不同，不能从系数直接推出总梯度范数；可确定的是目标引入了样本长度相关权重。固定常数归一化去掉这一来源。

## 9. Kimi 的均值基线为什么只是 τlogZ 的近似？

$Z=E_{\pi_i}\exp(r/\tau)$。展开得 $\tau\log Z=E[r]+\mathrm{Var}(r)/(2\tau)+\dots$，有限τ时不等于均值；实践用样本均值替代还带来估计与相关性问题。平方代理损失在旧策略分布下采样，下降方向为(r−b−τlog比值)∇logπθ，不是无条件等于精确 KL 正则目标的梯度。

## 10. Kimi 一组长度100、200、300，错误答案长度100是否有正长度奖励？

λ分别为0.5、0、−0.5。错误答案用min(0,λ)，因此长度100和200的长度奖励都为0，长度300为−0.5。门槛是最短与最长的中点，不是长度均值；所有长度相同时，Kimi 原论文把长度奖励都设为0，避免分母为0。

## 11. R1 的80万蒸馏数据和 Qwen3 的3995题可以直接比较数据效率吗？

不能。前者是约60万推理加20万非推理样本的监督数据规模；后者是推理 RL 的 query-verifier pairs，可被反复采样，且此前已有预训练和冷启动。比较还需给出基座、总生成 token、验证和筛选成本、训练轮数、任务分布与测试预算。

## 12. 为什么第41页“best-of-8”需要额外说明？

Kimi 原论文用10次高温 SFT 采样估计难度；8次过滤针对不带 CoT 的直接猜答案，避免容易碰巧获奖的题。它不等价于删除所有能在8条推理链中解出的题。原页保留，中文解释区分了两种筛选目的。

## 13. 看见模型经 RL 后多写“等等”且长度增加，应如何验证能力是否改善？

至少比较独立题目上、固定计算预算的正确率，检查基座是否已有相似行为，并分别统计有效纠正、冗余、正确与错误回答长度。文字样例、长度相关性与可重复的能力提升是不同证据层级，不能互相替代。

## 14. 代码 agent 的测试分数突然暴涨，最先检查什么？

查看轨迹与环境约束，排查恢复远端仓库、获取参考修复、修改测试或改变测试运行方式等捷径。第60页给出真实 reward hacking 例子；启发式 blocker 对已发现路径有效，不代表所有漏洞已消失。必

须同时报告 agent 框架、测试集、工具权限和独立审查。

### 15. 为何不能把前半讲的 contextual bandit 概括直接套到多轮工具 agent？

静态题目可以把完整答案当一个动作，前缀形式仍有 token 状态；工具 agent 则通过动作改变外部环境并获得新观察，状态转移、失败恢复与信用分配更复杂。环境生成的观察也不应当成策略采样动作来计算 log概率损失。

# 参考资料

**[slides] Stanford CS336 Spring 2026 第16讲官方原课件；原页为本讲首要依据**

https://raw.githubusercontent.com/stanford-cs336/lectures/main/lecture_16.pdf

**[captions] 第三方保存的 Spring 2026 第16讲录像字幕，仅作课堂定位，非教师审校稿**

https://jingzechen.github.io/assets/courses/stanford-cs336-s26/lectures/016/captions.source.srt

**[ppo] OpenAI Spinning Up：PPO 官方教学文档**

https://spinningup.openai.com/en/latest/algorithms/ppo.html

**[alpaca] AlpacaFarm PPO 原作者代码，固定到课件给定提交30717dda**

https://github.com/tatsu-lab/alpaca_farm/blob/30717ddae735365de756ee2085191b491a71788d/src/alpaca_farm/rl/ppo_trainer.py

**[nano] nano-aha-moment：课件引用的简化 GRPO 实现仓库；当前仓库不保证等同原截图版本**

https://github.com/McGill-NLP/nano-aha-moment

**[math] DeepSeekMath：GRPO 原论文，v3**

https://arxiv.org/html/2402.03300v3

**[dr] Understanding R1-Zero-Like Training: A Critical Perspective；Dr. GRPO 原论文，v2**

https://arxiv.org/html/2503.20783v2

**[r1] DeepSeek-R1 原始技术报告，v1；用于核对课件所述训练流程**

https://arxiv.org/html/2501.12948v1

**[kimi] Kimi k1.5: Scaling Reinforcement Learning with LLMs；原始技术报告，v1**

https://arxiv.org/html/2501.12599v1

**[s1] s1: Simple test-time scaling；原作者论文**

https://arxiv.org/abs/2501.19393

### [qwen] Qwen3 Technical Report；原始技术报告，v1

https://arxiv.org/html/2505.09388v1

### [coder] Qwen3-Coder-Next Technical Report；原始技术报告，v1

https://arxiv.org/html/2603.00729v1